

Hướng Dẫn Thực Hành - Lập Trình Web Với Node.js

Tài liệu hướng dẫn thực hành dành cho sinh viên. Mỗi bài gồm: mục tiêu, hướng dẫn từng bước, cách chạy, kết quả mong đợi và lỗi thường gặp.

Yêu cầu trước khi bắt đầu: Đã cài đặt Node.js (phiên bản 18 trở lên). Kiểm tra bằng lệnh:

```
node -v
npm -v
```

Nếu chưa cài, tải tại: <https://nodejs.org>

Mục Lục

- **[Chương 0: Chuẩn Bị Môi Trường Phát Triển](#)**
 - [0.1: Cài Đặt Node.js](#)
 - [0.2: Cài Đặt Visual Studio Code](#)
 - [0.3: Cài Đặt Postman / Thunder Client \(Test API\)](#)
 - [0.4: Tạo Tài Khoản MongoDB Atlas](#)
 - [0.5: Các Lệnh Terminal Cơ Bản](#)
 - [0.6: Khắc Phục Sự Cố Thường Gặp](#)
- **[Chương 1: Giới Thiệu Node.js](#)**
 - [Bài 1.4: Tạo HTTP Server Cơ Bản](#)
- **[Chương 2: Các Module Built-in Của Node.js](#)**
 - [Bài 2.2-os: Module OS - Thông Tin Hệ Điều Hành](#)
 - [Bài 2.2-path: Module Path - Xử Lý Đường Dẫn](#)
 - [Bài 2.2-fs: Module File System - Đọc/Ghi File](#)
 - [Bài 2.th-01: Thực Hành Tổng Hợp - Đọc JSON & Ghi Báo Cáo](#)
- **[Chương 3: Express.js Cơ Bản](#)**
 - [Bài 3.1: Node Modules & NPM](#)
 - [Bài 3.3: Express, Morgan & Các Loại Tham Số Request](#)
 - [Bài 3.th: Thực Hành - Xây Dựng RESTful API CRUD Sinh Viên](#)
- **[Chương 4: Kiến Trúc MVC](#)**
 - [Bài 4.0: So Sánh HTTP Thuần vs Express](#)
 - [Bài 4.4-mvc: Giới Thiệu Kiến Trúc MVC](#)
 - [Bài 4.4.thuchanh: Thực Hành - API Users Đơn Giản \(Chưa MVC\)](#)
 - [Bài 4.4.thuchanh-2: Thực Hành - CRUD Đầy Đủ Với MVC \(Nhiều Resource\)](#)
- **[Chương 5: Template Engine - EJS](#)**
 - [Bài 5.2: Ứng Dụng Web Với EJS](#)
- **[Chương 6: Middleware](#)**
 - [Bài 6.2-code-log: Middleware Ghi Log Request](#)
 - [Bài 6.2-code-role-admin: Middleware Kiểm Tra Quyền \(Authorization\)](#)
- **[Chương 7: MongoDB & Mongoose](#)**
 - [Bài 7.3: Giới Thiệu Mongoose - Kết Nối MongoDB](#)
 - [Bài 7.4: Thực Hành - CRUD Product Với Mongoose \(MVC\)](#)
- **[Chương 9: MongoDB Nâng Cao](#)**
 - [Bài 9.4: Query Operators - Toán Tử Truy Vấn MongoDB](#)

- [Bài 9.th: Thực Hành - Explain Query, Index & Phân Trang](#)
- [Chương 10: Xác Thực & Phiên Làm Việc](#)
 - [Bài 10.2: Mã Hóa Mật Khẩu Với Bcrypt](#)
 - [Bài 10.3: Session & Middleware Xác Thực](#)

Quy Ước Ký Hiệu Trong Tài Liệu

Để bạn dễ phân biệt việc nào làm ở Terminal, việc nào sửa trong code, mỗi bước thực hành sẽ được gắn 1 trong các ký hiệu sau:

Ký hiệu	Ý nghĩa	Bạn làm gì?
 TERMINAL	Lệnh chạy trong Terminal / PowerShell / Command Prompt	Mở Terminal (hoặc Terminal tích hợp trong VS Code bằng Ctrl + `), gõ/dán lệnh, nhấn Enter
 TẠO FILE	Tạo file MỚI trong VS Code	Trong VS Code: chuột phải vào thư mục → New File → đặt tên → dán nội dung
 SỬA FILE	Sửa nội dung file CÓ SẴN	Mở file trong VS Code, sửa theo hướng dẫn, nhấn Ctrl+S / Cmd+S để lưu
 TRÌNH DUYỆT	Mở/test trên trình duyệt web	Chrome/Firefox/Edge → nhập URL → xem kết quả

Cách nhận biết qua đoạn code:

Cú pháp	Loại	Bạn làm gì?
<code>```bash</code>	Lệnh Terminal	Chạy trong Terminal
<code>```javascript</code>	Code JavaScript	Dán vào file .js
<code>```json</code>	Cấu hình JSON	Dán vào package.json, .json...
<code>```html</code>	Mã HTML	Dán vào file .html hoặc .ejs
<code>```css</code>	Mã CSS	Dán vào file .css
<code>```text</code> (không có ngôn ngữ)	Nội dung file text thuần	Dán vào file tương ứng (.env, .gitignore...)

Mẹo: Khi gõ lệnh Terminal phức tạp nhiều dòng có dấu `\` ở cuối (như `curl`), bạn có thể copy toàn bộ - Terminal sẽ hiểu là 1 lệnh. Trên Windows PowerShell, đôi khi cần thay `\` ở cuối dòng bằng ``` (backtick).

Chương 0: Chuẩn Bị Môi Trường Phát Triển

Mục đích chương này: Hướng dẫn cài đặt đầy đủ các phần mềm cần thiết **TRƯỚC KHI** bắt đầu học. Bạn chỉ cần làm 1 lần. Nếu đã cài rồi có thể bỏ qua.

0.1: Cài Đặt Node.js

Node.js là gì? Node.js là môi trường giúp chạy mã JavaScript trên máy tính (server) thay vì chỉ chạy trong trình duyệt. Khi cài Node.js, bạn cũng được cài kèm **NPM** (Node Package Manager) - công cụ quản lý thư viện.

Cài trên Windows

1. Truy cập trang chủ: <https://nodejs.org>
2. Trên trang chủ có 2 nút tải:
 - o **LTS** (Long Term Support) - Phiên bản ổn định, **khuyến dùng cho sinh viên**.
 - o **Current** - Phiên bản mới nhất, có thể còn lỗi.
3. Bấm vào nút **LTS** để tải file `.msi` (file cài đặt cho Windows).
4. Mở file `.msi` vừa tải về, làm theo các bước:
 - o Bấm **Next**
 - o Đọc và đồng ý license, bấm **Next**
 - o Để mặc định đường dẫn cài đặt (`C:\Program Files\nodejs\`), bấm **Next**
 - o Bấm **Next** ở các bước tiếp theo (giữ nguyên mặc định)
 - o Tại bước "Tools for Native Modules": **KHÔNG cần tick chọn** (sinh viên mới học chưa cần). Bấm **Next**
 - o Bấm **Install** (có thể cần nhập mật khẩu admin)
 - o Đợi cài xong, bấm **Finish**
5. **Khởi động lại** máy tính để áp dụng biến môi trường PATH (quan trọng!).

Cài trên macOS

Cách 1 - Cài qua file `.pkg` (đơn giản nhất):

1. Truy cập <https://nodejs.org>
2. Bấm nút **LTS** để tải file `.pkg`
3. Mở file `.pkg`, làm theo hướng dẫn (bấm Continue -> Continue -> Agree -> Install)
4. Nhập mật khẩu máy Mac khi được hỏi

Cách 2 - Cài qua Homebrew (nếu đã có Homebrew):

```
brew install node
```

Kiểm tra cài đặt thành công

Mở Terminal (macOS) hoặc Command Prompt / PowerShell (Windows), gõ:

```
node -v
npm -v
```

Kết quả mong đợi (số phiên bản có thể khác nhưng phải hiện ra):

v20.11.0
10.2.4

Nếu báo lỗi 'node' is not recognized (Windows): Bạn đã quên khởi động lại máy. Hoặc PATH chưa được cấu hình - mở "Environment Variables" -> Path -> thêm `C:\Program Files\nodejs\`.

Yêu cầu phiên bản: Tài liệu này dùng Node.js **18 trở lên**. Phiên bản LTS hiện tại (20+) là OK.

0.2: Cài Đặt Visual Studio Code

VS Code là gì? Là phần mềm soạn thảo code miễn phí của Microsoft, được dùng phổ biến nhất hiện nay cho JavaScript/Node.js.

Tải và cài

1. Truy cập: <https://code.visualstudio.com>
2. Trang web tự động nhận diện hệ điều hành và hiện nút tải phù hợp. Bấm **Download**.
3. **Windows:** Mở file `.exe`, làm theo hướng dẫn. Ở bước "Select Additional Tasks", nên tick chọn:
 - ☒ Add "Open with Code" action to Windows Explorer file context menu
 - ☒ Add "Open with Code" action to Windows Explorer directory context menu
 - ☒ Add to PATH (mặc định đã tick)
4. **macOS:** Giải nén file `.zip`, kéo Visual Studio Code.app vào thư mục **Applications**.

Extension nên cài (rất hữu ích)

Mở VS Code -> Bấm icon Extensions ở thanh bên trái (4 ô vuông) hoặc nhấn `Ctrl+Shift+X` (Windows) / `Cmd+Shift+X` (Mac). Tìm và cài các extension sau:

Extension	Tác giả	Tác dụng
ESLint	Microsoft	Báo lỗi cú pháp JavaScript ngay khi gõ
Prettier - Code formatter	Prettier	Tự động format code đẹp khi save (Shift+Alt+F)
Thunder Client	Ranga Vadhineni	Test API ngay trong VS Code (thay Postman)
MongoDB for VS Code	MongoDB	Xem dữ liệu MongoDB trực tiếp trong VS Code
Vietnamese Language Pack (tùy chọn)	Microsoft	Đổi giao diện VS Code sang tiếng Việt
Material Icon Theme (tùy chọn)	Philipp Kief	Icon đẹp hơn cho file/folder

Mở Terminal tích hợp trong VS Code

Trong VS Code, nhấn `Ctrl + `` (phím dấu huyền, ở góc trên bên trái bàn phím) để mở Terminal. Terminal này là `cmd` / `powershell` (Windows) hoặc `zsh` / `bash` (macOS).

Mẹo: Bạn có thể chạy `node`, `npm` ngay trong Terminal này, không cần mở Terminal riêng.

0.3: Cài Đặt Postman / Thunder Client (Test API)

Khi xây dựng API, bạn cần công cụ để **gửi request** kiểm tra. Có 2 lựa chọn:

Lựa chọn 1: Thunder Client (khuyên dùng - nhẹ, tích hợp VS Code)

Đã hướng dẫn cài ở mục 0.2. Sau khi cài:

1. Bấm icon **tia sét** ở thanh bên trái VS Code.
2. Bấm **New Request** -> Nhập URL (vd: `http://localhost:3000`) -> Bấm **Send**.

Lựa chọn 2: Postman (chuyên nghiệp, nhiều tính năng)

1. Truy cập: <https://www.postman.com/downloads>
2. Tải bản phù hợp (Windows/macOS), cài đặt.
3. Mở Postman, có thể bấm **Skip and go to the app** nếu không muốn đăng ký.
4. Bấm **New** -> **HTTP Request** -> Nhập URL -> Bấm **Send**.

Lựa chọn 3: Lệnh `curl` trong Terminal

Đã có sẵn trên macOS và Windows 10/11. Cú pháp ví dụ:

```
curl http://localhost:3000
curl -X POST http://localhost:3000 -H "Content-Type: application/json" -d
'{"name": "tuan"}'
```

Trong tài liệu này, các ví dụ test API sẽ dùng `curl` cho đồng nhất. Bạn có thể dùng Thunder Client/Postman thay thế.

0.4: Tạo Tài Khoản MongoDB Atlas

Chỉ cần làm khi học đến Chương 7 trở đi. Có thể bỏ qua mục này lúc đầu.

MongoDB Atlas là gì? Là dịch vụ MongoDB chạy trên đám mây (cloud) do hãng MongoDB cung cấp. Có **gói miễn phí (M0)** đủ dùng cho học tập.

Các bước thực hiện

Bước 1: Đăng ký tài khoản

1. Truy cập: <https://cloud.mongodb.com>
2. Bấm **Try Free** -> Nhập email, mật khẩu (hoặc đăng nhập bằng Google).
3. Xác minh email nếu được yêu cầu.

Bước 2: Tạo Cluster (cụm database)

1. Chọn loại **M0 FREE** (miễn phí mãi mãi - 512MB).
2. Provider: **AWS** (mặc định).
3. Region: Chọn nơi gần Việt Nam, vd: **Singapore (ap-southeast-1)** để truy cập nhanh.
4. Cluster Name: Để mặc định `Cluster0` hoặc đổi tên tùy ý.
5. Bấm **Create Deployment**. Chờ 1-3 phút cluster được tạo.

Bước 3: Tạo Database User Khi cluster đang tạo, hiện popup "Connect to Cluster0":

1. Username: vd `admin` (hoặc tùy ý).

2. Password: Bấm **Autogenerate Secure Password** rồi **Copy** lưu lại (RẤT QUAN TRỌNG - lát nữa cần dùng).
3. Bấm **Create Database User**.

Bước 4: Cho phép IP truy cập (Network Access)

1. Menu bên trái -> **Network Access** -> **Add IP Address**.
2. Có 2 lựa chọn:
 - o **ADD CURRENT IP ADDRESS** - Chỉ cho phép IP hiện tại của bạn (an toàn nhưng IP thay đổi khi đổi mạng).
 - o **ALLOW ACCESS FROM ANYWHERE** (IP `0.0.0.0/0`) - Cho phép mọi IP truy cập (**nhận chọn cho việc học tập** để khỏi gặp lỗi khi đổi mạng).
3. Bấm **Confirm**.

Bước 5: Lấy Connection String

1. Menu bên trái -> **Database** -> Bấm **Connect** trên cluster của bạn.
2. Chọn **Drivers** (loại kết nối qua code).
3. Chọn Driver: **Node.js**, Version: mới nhất.
4. Copy connection string, dạng:

```
mongodb+srv://admin:<db_password>@cluster0.xxxxx.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0
```

5. Thay `<db_password>` bằng mật khẩu đã lưu ở Bước 3.
6. Lưu chuỗi này để dùng cho biến `MONGODB_URI` trong file `.env` của các bài Chương 7+.

Lưu ý bảo mật: KHÔNG share connection string này lên Git, Facebook... vì người khác có thể truy cập database của bạn.

0.5: Các Lệnh Terminal Cơ Bản

Sinh viên mới học thường ngại Terminal. Thực ra chỉ cần biết vài lệnh cơ bản:

Lệnh dùng chung (cả Windows PowerShell và macOS Terminal)

Lệnh	Ý nghĩa	Ví dụ
<code>pwd</code>	Hiển thị thư mục hiện tại (Print Working Directory)	<code>pwd</code>
<code>ls</code>	Liệt kê file/thư mục trong thư mục hiện tại	<code>ls</code>
<code>cd <thư mục></code>	Chuyển vào thư mục con (Change Directory)	<code>cd my-project</code>
<code>cd ..</code>	Quay lại thư mục cha (lùi 1 cấp)	<code>cd ..</code>
<code>cd ~</code>	Về thư mục home của user	<code>cd ~</code>
<code>mkdir <tên></code>	Tạo thư mục mới	<code>mkdir my-project</code>
<code>clear</code> (Mac) / <code>cls</code> (Win)	Xóa màn hình terminal	<code>clear</code>

Ctrl + C	Dừng chương trình đang chạy	(nhấn 2 phím cùng lúc)
↑ / ↓	Lấy lại lệnh đã gõ trước đó	(mũi tên lên/xuống)
Tab	Tự động hoàn thành tên file/thư mục	gõ cd my rồi Tab

Riêng Windows Command Prompt (cmd) khác chút

Lệnh Windows cmd	Tương đương Mac/Linux
dir	ls
cls	clear
cd /d D:\	Chuyển sang ổ D

Mẹo: Nếu dùng Windows, khuyên dùng **PowerShell** hoặc **Git Bash** thay vì cmd cũ vì hỗ trợ nhiều lệnh giống macOS/Linux hơn.

Ví dụ workflow tạo project Node.js

```
# 1. Về Desktop
cd ~/Desktop      # Mac
cd %USERPROFILE%\Desktop  # Windows

# 2. Tạo thư mục project
mkdir my-first-app

# 3. Vào thư mục vừa tạo
cd my-first-app

# 4. Khởi tạo project Node.js
npm init -y

# 5. Mở thư mục này trong VS Code
code .
```

code . - Mở thư mục hiện tại (dấu **.** nghĩa là "ở đây") trong VS Code. Lệnh này chỉ hoạt động nếu khi cài VS Code đã tick chọn "Add to PATH".

0.6: Khắc Phục Sự Cố Thường Gặp

Vấn đề	Nguyên nhân	Cách khắc phục
'node' is not recognized as an internal or external command (Windows)	PATH chưa cấu hình hoặc chưa khởi động lại máy	Khởi động lại máy. Nếu vẫn lỗi: cài lại Node.js, tick chọn "Add to PATH"

command not found: node (macOS)	Tương tự trên	Cài lại Node.js từ file .pkg, mở Terminal MỚI
npm install rất chậm hoặc lỗi mạng	Mạng yếu hoặc bị chặn npm registry	Đổi sang dùng pnpm hoặc yarn, hoặc đổi registry: npm config set registry https://registry.npmirror.com
EACCES: permission denied khi npm install (macOS)	Cài npm với sudo trước đó nên phân quyền sai	Đừng dùng sudo npm install. Sửa quyền: sudo chown - R \$(whoami) ~/.npm
Port 3000 đang bị chiếm (EADDRINUSE)	Có chương trình khác chiếm port 3000, hoặc chưa Ctrl+C server cũ	Đổi sang port khác (3001, 8080...), hoặc tìm và kill process: macOS: lsof -i :3000 rồi kill -9 <PID>. Windows: netstat -ano findstr :3000 rồi taskkill /PID <PID> /F
VS Code không nhận node trong Terminal tích hợp	Terminal mở trước khi cài Node	Đóng VS Code và mở lại
Code copy từ tài liệu bị lỗi cú pháp (dấu nháy lạ)	PDF/Word đổi nháy " thành nháy cong "	Gõ tay lại các dấu nháy, hoặc copy từ bản Markdown gốc

Chương 1: Giới Thiệu Node.js

Bài 1.4: Tạo HTTP Server Cơ Bản

Mục tiêu

- Tạo project Node.js với `npm init`
- Tạo HTTP server đơn giản
- Phân biệt 2 hệ thống module: **ES Modules** (`import`) và **CommonJS** (`require`)

Bước 1: Tạo thư mục project (TERMINAL)

Mở Terminal (hoặc Command Prompt trên Windows), chạy các lệnh sau:

```
mkdir my-node-project    # Tạo thư mục mới
cd my-node-project       # Vào trong thư mục vừa tạo
```

Bước 2: Khởi tạo project Node.js (TERMINAL)

```
npm init -y
```

Lệnh này tạo file `package.json` - file cấu hình chính của project. Flag `-y` nghĩa là chấp nhận tất cả giá trị mặc định.

Bước 3: Bật chế độ ES Modules (SỬA FILE)

Mở file `package.json` trong VS Code, **thêm** dòng `"type": "module"` . Sau khi sửa file trông như sau:

```
{
  "name": "my-node-project",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "type": "module"
}
```

Tại sao cần `"type": "module"` ? Mặc định Node.js dùng CommonJS (`require`). Thêm dòng này để chuyển sang ES Modules (`import`) - cú pháp hiện đại hơn và là tiêu chuẩn của JavaScript.

Bước 4: Tạo HTTP Server với ES Modules (TẠO FILE)

Tạo file mới `index.js` trong thư mục `my-node-project/` , dán nội dung sau:

```
// Import module 'http' có sẵn trong Node.js (built-in module).
// Prefix 'node:' (tùy chọn) khẳng định rõ đây là module của Node, không phải
package từ npm.
import http from 'node:http'

// http.createServer() tạo ra một HTTP server.
// Tham số là một CALLBACK FUNCTION – hàm này sẽ tự động được gọi MỖI KHI có client
gửi request tới.
// - req (request): Object chứa thông tin client gửi lên (URL, method, headers,
body...)
// - res (response): Object dùng để gửi dữ liệu phản hồi về cho client
const server = http.createServer((req, res) => {
  // res.end(data) gửi nội dung 'data' về client VÀ kết thúc response (đóng kết
nối).
  // Nếu không gọi res.end(), trình duyệt sẽ chờ mãi không nhận được phản hồi.
  res.end('Hello world from Node.js!')
})

// server.listen(port, callback) bảo server "mở cổng" lắng nghe các kết nối đến.
// - Port 3000: là số "cổng" để client kết nối vào (có thể chọn 3001, 8080... nếu
3000 bị chiếm)
// - Callback: hàm chạy MỘT LẦN khi server đã khởi động thành công, dùng để in thông
báo
server.listen(3000, () => {
  console.log('Server is running ...')
})
```

Giải thích:

- `import http from 'node:http'` - Import module `http` có sẵn trong Node.js. Prefix `node:` cho biết đây là module built-in.
- `http.createServer()` - Tạo server, nhận callback với 2 tham số: `req` (request - yêu cầu từ client) và `res` (response - phản hồi từ server).
- `res.end('Hello world from Node.js!')` - Gửi nội dung text về cho client và kết thúc response.
- `server.listen(3000, callback)` - Server lắng nghe trên port 3000. Callback chạy khi server khởi động thành công.

Bước 5: Chạy server (🖥️ TERMINAL)

```
node index.js
```

Kết quả mong đợi trên Terminal:

```
Server is running ...
```

Test trên trình duyệt (🌐 TRÌNH DUYỆT): Mở trình duyệt, truy cập địa chỉ: `http://localhost:3000`

Trình duyệt sẽ hiển thị: **Hello world from Node.js!**

Dừng server: Nhấn `Ctrl + C` trong Terminal.

Bước 6: So sánh với CommonJS (🔗 TẠO FILE - Tham khảo)

Tạo file mới `index.cjs` trong cùng thư mục, dán nội dung sau:

```
// require() là cú pháp import của CommonJS – hệ thống module CŨ của Node.js.
// Không cần prefix 'node:' vì CommonJS không hỗ trợ.
const http = require('http')

// Phần tạo server giống hệt như ES Module bên trên.
const server = http.createServer((req, res) => {
  res.end('Hello world from Node.js!')
})

server.listen(3000, () => {
  console.log('Server is running ...')
})

// __dirname và __filename là 2 BIẾN ĐẶC BIỆT chỉ có trong CommonJS:
// __dirname = đường dẫn thư mục chứa file đang chạy (vd: /Users/ten/my-project)
// __filename = đường dẫn đầy đủ tới file đang chạy (vd: /Users/ten/my-project/index.cjs)
// Trong ES Module 2 biến này KHÔNG CÓ SẴN – phải tạo lại bằng import.meta.url (sẽ học ở Chương 2).
console.log(__dirname)
console.log(__filename)
```

Chạy file này (🖥️ TERMINAL):

```
# Nhớ dừng server cũ (index.js) bằng Ctrl+C trước nếu đang chạy
node index.cjs
```

Kết quả mong đợi:

```
/duong/dan/den/my-node-project
/duong/dan/den/my-node-project/index.cjs
Server is running ...
```

So sánh ES Modules vs CommonJS:

	ES Modules	CommonJS
Import	<code>import http from 'node:http'</code>	<code>const http = require('http')</code>
Extension	<code>.js</code> (khi có <code>"type": "module"</code>)	<code>.cjs</code> hoặc <code>.js</code> (mặc định)
<code>__dirname</code> , <code>__filename</code>	Không có	Có sẵn
Tiêu chuẩn	JavaScript hiện đại	Truyền thống của Node.js

Lưu ý: File `.cjs` luôn được Node.js xử lý theo CommonJS, bất kể cấu hình `"type"` trong `package.json`.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
SyntaxError: Cannot use import statement outside a module	Thiếu "type": "module" trong package.json	Thêm "type": "module" vào package.json
EADDRINUSE: address already in use :::3000	Port 3000 đang bị chương trình khác chiếm	Tắt chương trình đang dùng port 3000 (có thể do chưa Ctrl+C server trước đó), hoặc đổi sang port khác (vd: 3001)
ReferenceError: __dirname is not defined	Dùng __dirname trong file ES Module	__dirname chỉ có trong CommonJS. Trong ES Module, sử dụng cách khác (sẽ học ở Chương 2)
ReferenceError: require is not defined in ES module scope	Dùng require() trong file ES Module	Đổi require() thành import

Chương 2: Các Module Built-in Của Node.js

Bài 2.2-os: Module OS - Thông Tin Hệ Điều Hành

Mục tiêu

- Sử dụng module `os` để lấy thông tin hệ điều hành
- Hiểu các hàm thường dùng: `platform()`, `arch()`, `hostname()`, `cpus()`, `totalmem()`

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 2.2-os
cd 2.2-os
npm init -y
```

Bước 1b: Bật ES Modules (📝 SỬA FILE)

Mở `package.json`, thêm dòng `"type": "module"`.

Bước 2: Viết code (📝 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
// Import module 'os' (Operating System) - module built-in cung cấp thông tin về hệ điều hành.
import os from 'node:os'

// os.platform() trả về tên hệ điều hành: 'darwin' (macOS), 'win32' (Windows), 'linux'
console.log('Platform: ', os.platform());

// os.arch() trả về kiến trúc CPU: 'x64' (Intel/AMD 64-bit), 'arm64' (Apple Silicon M1/M2/M3)
console.log('Architecture: ', os.arch());

// os.hostname() trả về tên máy tính (vd: "MacBook-Pro.local", "DESKTOP-ABC123")
console.log('Hostname: ', os.hostname());

// os.homedir() trả về đường dẫn thư mục home của user đang đăng nhập
// Mac/Linux: /Users/tenuser   Windows: C:\Users\tenuser
console.log('Hom Directory: ', os.homedir());

// os.cpus() trả về mảng các object mô tả từng nhân CPU.
// .length cho biết số nhân (vd: 8 nhân, 16 nhân...)
console.log("Number of CPU cores: ", os.cpus().length);

// os.totalmem() trả về tổng RAM tính bằng BYTE (số rất lớn, vd 17179869184).
// Chia 3 lần cho 1024 để đổi: byte -> KB -> MB -> GB cho dễ đọc.
console.log("Total memory: ", os.totalmem() / 1024 / 1024 / 1024, " GB");
```

```
// os.networkInterfaces() trả về thông tin các card mạng (IP, MAC address...)
// Hữu ích khi muốn biết IP máy đang dùng để chia sẻ trong mạng LAN.
console.log("Nework details: ", os.networkInterfaces());
```

Giải thích các hàm:

Hàm	Ý nghĩa	Ví dụ kết quả
os.platform()	Hệ điều hành đang chạy	darwin (macOS), win32 (Windows), linux
os.arch()	Kiến trúc CPU	x64, arm64
os.hostname()	Tên máy tính	MacBook-Pro.local
os.homedir()	Thư mục home của user	/Users/ten (macOS), C:\Users\ten (Windows)
os.cpus().length	Số lõi CPU	8
os.totalmem()	Tổng RAM (đơn vị byte)	Chia 3 lần cho 1024 để đổi sang GB
os.networkInterfaces()	Thông tin các card mạng	Object chứa IP, MAC address...

Bước 3: Chạy chương trình (TERMINAL)

```
node index.js
```

Kết quả mong đợi (giá trị khác nhau tùy máy):

```
Platform: darwin
Architecture: arm64
Hostname: MacBook-Pro.local
Hom Directory: /Users/ten
Number of CPU cores: 8
Total memory: 16 GB
Nework details: { lo0: [...], en0: [...] }
```

Bài 2.2-path: Module Path - Xử Lý Đường Dẫn

Mục tiêu

- Sử dụng module `path` để thao tác với đường dẫn file
- Hiểu cách lấy `__filename` và `__dirname` trong ES Modules
- Các hàm: `dirname()`, `join()`, `extname()`, `resolve()`

Bước 1: Tạo project (TERMINAL)

```
mkdir 2.2-path
cd 2.2-path
npm init -y
```

Bước 1b: Bật ES Modules (📄 SỬA FILE)

Thêm `"type": "module"` vào `package.json`.

Bước 1c: Tạo thư mục test và file test (💻 TERMINAL)

```
# macOS / Linux:
mkdir test-folder
touch test-folder/test.txt
```

Trên Windows PowerShell dùng:

```
mkdir test-folder
New-Item test-folder\test.txt
```

Hoặc nếu dùng Command Prompt cũ: `mkdir test-folder` rồi `echo. > test-folder\test.txt`

Bước 2: Viết code (📄 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
// Import hàm fileURLToPath từ module 'url' - dùng để chuyển URL thành đường dẫn file.
import { fileURLToPath } from 'url';
// Import module 'path' - chuyên xử lý các thao tác với đường dẫn (nối, tách, đổi đuôi...)
import path from 'path';

// import.meta.url là URL của file hiện tại, dạng: 'file:///Users/ten/2.2-path/index.js'
// fileURLToPath() chuyển URL đó thành đường dẫn thường: '/Users/ten/2.2-path/index.js'
// → Đây là CÁCH THAY THẾ cho __filename của CommonJS trong môi trường ES Module.
const filePath = fileURLToPath(import.meta.url);
console.log(filePath); // In ra: /duong/dan/index.js

// path.dirname() lấy phần "thư mục cha" từ một đường dẫn file.
// Vd: '/Users/ten/2.2-path/index.js' → '/Users/ten/2.2-path'
// → Đây là cách THAY THẾ cho __dirname của CommonJS trong ES Module.
const dirPath = path.dirname(filePath)
console.log(dirPath)

// path.join() nối các phần đường dẫn lại, TỰ ĐỘNG dùng '/' (Mac/Linux) hay '\' (Windows).
// Nên LUÔN dùng path.join thay vì tự nối chuỗi '+' để code chạy được trên mọi hệ điều hành.
const filePath_2 = path.join(dirPath, "index.js");
console.log(filePath_2);

// path.extname() lấy phần đuôi mở rộng của file (vd: '.js', '.txt', '.png').
```

```

console.log(path.extname(filePath_2));

// 'test-folder' là đường dẫn TƯƠNG ĐỐI (relative) – phụ thuộc vào nơi chạy lệnh
node.
const testFolderPath = 'test-folder';
const testFilePath = path.join(testFolderPath, "test.txt");

// path.resolve() chuyển đường dẫn tương đối → đường dẫn TUYỆT ĐỐI (absolute).
// Đường dẫn tuyệt đối bắt đầu từ gốc ổ đĩa, vd: '/Users/ten/duan/test-
folder/test.txt'
const absolutePath = path.resolve(testFilePath);
console.log(absolutePath)

```

Giải thích:

- **import.meta.url** - Trong ES Module, đây là cách lấy URL của file hiện tại (dạng `file:///duong/dan/index.js`).
- **fileURLToPath(import.meta.url)** - Chuyển URL thành đường dẫn file bình thường, thay thế cho `__filename` (chỉ có trong CommonJS).
- **path.dirname(filePath)** - Lấy đường dẫn thư mục chứa file, thay thế cho `__dirname`.
- **path.join(dirPath, "index.js")** - Nối các phần đường dẫn lại với nhau. Tự động thêm `/` hoặc `\` tùy hệ điều hành.
- **path.extname(filePath_2)** - Lấy phần mở rộng của file (vd: `.js`, `.txt`).
- **path.resolve(testFilePath)** - Chuyển đường dẫn tương đối thành đường dẫn tuyệt đối.

Bước 3: Chạy chương trình (🖥️ TERMINAL)

```
node index.js
```

Kết quả mong đợi:

```

/duong/dan/2.2-path/index.js
/duong/dan/2.2-path
/duong/dan/2.2-path/index.js
.js
/duong/dan/2.2-path/test-folder/test.txt

```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ReferenceError: __dirname is not defined	Dùng <code>__dirname</code> trong ES Module	Dùng <code>path.dirname(fileURLToPath(import.meta.url))</code> thay thế
ReferenceError: __filename is not defined	Dùng <code>__filename</code> trong ES Module	Dùng <code>fileURLToPath(import.meta.url)</code> thay thế

Bài 2.2-fs: Module File System - Đọc/Ghi File

Mục tiêu

- Sử dụng module `fs` để đọc file
- Hiểu sự khác nhau giữa: đồng bộ (sync), bất đồng bộ callback, Promise, và Promise.all
- Hiểu khái niệm **blocking** vs **non-blocking**

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 2.2-fs
cd 2.2-fs
npm init -y
```

Bước 1b: Bật ES Modules (📝 SỬA FILE)

Thêm `"type": "module"` vào `package.json`.

Bước 1c: Tạo 3 file text để test (🖥️ TERMINAL)

```
echo "DAY LA FILE 1." > file1.txt
echo "DAY LA FILE 2" > file2.txt
echo "DAY LA FILE 3." > file3.txt
```

Hoặc bạn có thể tự tạo 3 file `file1.txt`, `file2.txt`, `file3.txt` trong VS Code và gõ nội dung tương ứng.

Bước 2: Đọc file đồng bộ - Synchronous (📝 TẠO FILE)

Tạo file mới `readFileSync.js`, dán nội dung:

```
// Import module 'fs' (File System) - cung cấp các hàm đọc/ghi file.
import fs from 'fs'

console.log('Đang đọc file đồng bộ ...');

// fs.readFileSync(path, encoding) là hàm ĐỒNG BỘ (Sync = Synchronous):
// - Chương trình DỪNG LẠI tại dòng này, chờ đọc xong file mới chạy tiếp.
// - 'utf-8' là encoding để trả về STRING (nếu thiếu, sẽ trả về Buffer - dạng nhị
phân khó đọc).
const data = fs.readFileSync('file1.txt', 'utf-8');
console.log('Data: ', data);

// Dòng này chỉ chạy SAU KHI file đã đọc xong (do tính chất đồng bộ).
console.log('=====');
```

Chạy (🖥️ TERMINAL):

```
node readFileSync.js
```

Kết quả:

```
Đang đọc file đồng bộ ...  
Data:  DAY LA FILE 1.  
=====
```

Giải thích: `readFileSync` là hàm **đồng bộ (blocking)** - chương trình dừng lại chờ đọc file xong rồi mới chạy dòng tiếp theo. Vì vậy kết quả luôn theo thứ tự: log -> data -> dấu `=` .

Bước 3: Đọc file bất đồng bộ với Callback (👉 TẠO FILE)

Tạo file mới `readFile.js` , dán nội dung:

```
import fs from 'fs'  
  
try {  
  console.log('Đang đọc file bất đồng bộ ...');  
  
  // fs.readFile(path, encoding, callback) là hàm BẤT ĐỒNG BỘ (non-blocking):  
  // - Node.js GIAO việc đọc file cho hệ điều hành rồi CHẠY TIẾP, không chờ.  
  // - Khi đọc file xong (có thể vài mili giây sau), callback (err, data) => {...}  
  mới được gọi.  
  // - err = thông tin lỗi nếu có (null nếu không lỗi)  
  // - data = nội dung file đọc được  
  fs.readFile('file1.txt', 'utf-8', (err, data) => {  
    console.log('Data: ', data);    // Dòng này chạy SAU dòng  
    console.log('=====') bên dưới!  
  })  
};  
} catch (error) {  
  // LƯU Ý: try/catch KHÔNG BẮT ĐƯỢC lỗi của callback bất đồng bộ.  
  // Để bắt lỗi của fs.readFile callback, phải kiểm tra biến `err` trong callback.  
  console.log('Lỗi đọc file: ', error.message)  
}  
  
// Dòng này chạy NGAY (không chờ fs.readFile xong) → in trước cả 'Data: ...'  
console.log('=====');
```

Chạy (🖥️ TERMINAL):

```
node readFile.js
```

Kết quả:

```
Đang đọc file bất đồng bộ ...  
=====  
Data:  DAY LA FILE 1.
```

Giải thích: `readFile` là hàm **bất đồng bộ (non-blocking)** - chương trình KHÔNG chờ đọc file xong mà chạy tiếp dòng `console.log('=====')` ngay. Khi đọc file xong, callback được gọi và in ra data. Vì

vậy dấu `=` = xuất hiện TRƯỚC data.

Đây là điểm khác biệt quan trọng nhất: Đồng bộ chờ xong mới chạy tiếp, bất đồng bộ chạy tiếp ngay không chờ.

Bước 4: Đọc file với `async/await` - `fs/promises` (📝 TẠO FILE)

Tạo file mới `readPromise.js` , dán nội dung:

```
// Import phiên bản 'fs/promises' - các hàm trả về Promise thay vì dùng callback.
// Promise giúp viết code bất đồng bộ gọn và dễ đọc hơn.
import fs from 'fs/promises'

// Hàm có từ khóa 'async' luôn trả về một Promise.
// Bên trong async function mới được dùng từ khóa 'await'.
async function docFile(fileName) {
  // 'await' tạm dừng hàm tại đây, chờ fs.readFile xong rồi mới chạy tiếp.
  // Quan trọng: await CHỈ tạm dừng hàm này, KHÔNG block các code khác đang chạy.
  const data = await fs.readFile(fileName, 'utf-8');
  // Template literal `${fileName}` cho phép chèn biến vào chuỗi.
  console.log(`Data of ${fileName}: `, data);
}

// IIFE (Immediately Invoked Function Expression) - hàm tự gọi ngay khi định nghĩa.
// Dùng async để có thể await bên trong (Node.js cũ chưa hỗ trợ top-level await).
(async () => {
  await docFile('file1.txt');
})();
```

Chạy (🖥️ TERMINAL):

```
node readPromise.js
```

Kết quả:

```
Data of file1.txt:  DAY LA FILE 1.
```

Giải thích: `fs/promises` trả về Promise thay vì dùng callback. Kết hợp với `async/await` giúp code gọn gàng và dễ đọc hơn. `await` sẽ chờ Promise hoàn thành, nhưng **không block** toàn bộ chương trình - chỉ tạm dừng trong hàm `async` đó.

Bước 5: Đọc nhiều file tuần tự với Promise chaining (📝 TẠO FILE)

Tạo file mới `promise.js` , dán nội dung:

```
import fs from 'fs/promises'

console.log('Đang đọc file bất đồng bộ ...');

// fs.readFile (bản promise) trả về một Promise.
// .then(callback) sẽ chạy callback KHI Promise hoàn thành thành công.
```

```

fs.readFile('file1.txt', 'utf-8')
  .then((data) => {
    // data là kết quả đọc file1.txt
    console.log('Data: ', data);
    // RETURN một Promise mới → cho phép chain (nối) .then() tiếp theo.
    return fs.readFile('./file2.txt', 'utf-8');
  })
  .then((data2) => {
    // data2 là kết quả của Promise return ở trên (đọc file2.txt)
    console.log('Data file 2: ', data2);
    return fs.readFile('./file3.txt', 'utf-8');
  })
  .then((data3) => {
    console.log('Data file 3: ', data3);
  })
  // .catch() bắt lỗi nếu BẤT KỲ Promise nào ở trên thất bại (file không tồn tại,
  // lỗi quyền...).
  .catch((error) => {
    console.error('Lỗi: ', error.message);
  })
  // .finally() LUÔN LUÔN chạy (dù thành công hay thất bại) – thường dùng để dọn
  // dẹp.
  .finally(() => {
    console.log('=====');
  })

```

Chạy (🖥️ TERMINAL):

```
node promise.js
```

Kết quả:

```

Đang đọc file bất đồng bộ ...
Data:  DAY LA FILE 1.
Data file 2:  DAY LA FILE 2
Data file 3:  DAY LA FILE 3.
=====

```

Giải thích: Đọc file1 xong -> đọc file2 -> đọc file3, theo thứ tự nối tiếp nhau (tuần tự). `.catch()` bắt lỗi nếu bất kỳ bước nào thất bại. `.finally()` luôn chạy dù thành công hay lỗi.

Bước 6: Đọc nhiều file song song với Promise.all (📝 TẠO FILE)

Tạo file mới `readPromiseAll.js`, dán nội dung:

```

import fs from 'node:fs/promises'

try {
  console.log('Đang đọc file với fs/promises ...');

```

```

    // performance.now() trả về thời điểm hiện tại (mili giây, có phần thập phân
    chính xác cao).
    // Dùng để đo thời gian thực thi của code.
    const startTime = performance.now();

    // Promise.all([p1, p2, p3]) chạy CÙNG LÚC (song song) tất cả Promise trong
    mảng.
    // Trả về 1 Promise mới, hoàn thành khi TẤT CẢ Promise con đều xong.
    // Kết quả là 1 mảng theo đúng thứ tự đầu vào: [kq1, kq2, kq3].
    // Destructuring [data, data2, data3] = mảng → tách thành 3 biến riêng để dễ
    dùng.
    const [data, data2, data3] = await Promise.all([
        fs.readFile('./file1.txt', 'utf-8'),
        fs.readFile('./file2.txt', 'utf-8'),
        fs.readFile('./file3.txt', 'utf-8')
    ])

    console.log('Data: ', data);
    console.log('Data file 2: ', data2);
    console.log('Data file 3: ', data3);

    const endTime = performance.now();
    // (endTime - startTime) = thời gian chạy (ms). .toFixed(3) làm tròn 3 số sau
    dấu phẩy.
    const duration = (endTime - startTime).toFixed(3);
    console.log('Execution time: ', duration);

} catch (error) {
    // Với async/await, try/catch BẮT ĐƯỢC lỗi của các await Promise (khác với
    callback).
    // Nếu BẤT KỲ file nào đọc lỗi → Promise.all reject → vào catch.
    console.log('Lỗi đọc file: ', error.message);
}

console.log('=====');
```

Chạy (🖥️ TERMINAL):

```
node readPromiseAll.js
```

Kết quả:

```

Đang đọc file với fs/promises ...
Data: DAY LA FILE 1.
Data file 2: DAY LA FILE 2
Data file 3: DAY LA FILE 3.
Execution time: 1.234
=====
```

Giải thích: `Promise.all()` chạy tất cả 3 thao tác đọc file **cùng lúc** (song song), không chờ file trước đọc xong mới đọc file sau. Kết quả trả về là mảng, dùng **destructuring** `[data, data2, data3]` để lấy từng giá trị. Nhanh hơn đáng kể so với đọc tuần tự khi có nhiều file.

Tóm tắt tiến hóa cách đọc file:

Cách	Ưu điểm	Nhược điểm
<code>readFileSync</code>	Đơn giản, dễ hiểu	Blocking - chương trình đứng chờ
<code>readFile (callback)</code>	Non-blocking	Code lồng nhau phức tạp (callback hell)
<code>fs/promises + async/await</code>	Gọn gàng, dễ đọc	Cần hiểu Promise
<code>Promise.all</code>	Song song, nhanh nhất	Nếu 1 file lỗi thì tất cả đều lỗi

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ENOENT: no such file or directory, open 'file1.txt'	File không tồn tại hoặc sai tên	Kiểm tra file tồn tại và đúng tên. Chú ý chạy lệnh node từ đúng thư mục chứa file
SyntaxError: Cannot use import statement outside a module	Thiếu "type": "module"	Thêm "type": "module" vào package.json
The "data" argument must be of type string...	Thiếu tham số 'utf-8' khi đọc file	Thêm 'utf-8' làm tham số thứ 2: <code>fs.readFile('file.txt', 'utf-8', ...)</code>
Warning: To load an ES module, set "type": "module"...	Dùng await ở top-level nhưng file không phải ES Module	Thêm "type": "module" vào package.json

Bài 2.th-01: Thực Hành Tổng Hợp - Đọc JSON & Ghi Báo Cáo

Mục tiêu

- Áp dụng `Promise.all` để đọc nhiều file JSON cùng lúc
- Sử dụng `JSON.parse()` để chuyển chuỗi JSON thành object JavaScript
- Sử dụng `fs.writeFile()` để ghi kết quả ra file
- Kết hợp các kiến thức đã học trong chương 2

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 2.th-01
cd 2.th-01
npm init -y
```

Bước 1b: Bật ES Modules (📝 SỬA FILE)

Thêm `"type": "module"` vào `package.json` .

Bước 2: Tạo dữ liệu - các file JSON danh sách sinh viên (📝 TẠO FILE)

Tạo file mới `lopA.json` trong thư mục project, dán nội dung:

```
{
  "students": [
    {
      "id": 1,
      "name": "tuan"
    },
    {
      "id": 2,
      "name": "vu"
    },
    {
      "id": 3,
      "name": "nguyễn"
    },
    {
      "id": 4,
      "name": "trần"
    }
  ]
}
```

Tạo file mới `lopB.json` , dán nội dung:

```
{
  "students": [
    {
      "id": 1,
      "name": "tuan"
    },
    {
      "id": 2,
      "name": "vu"
    },
    {
      "id": 3,
      "name": "nguyễn"
    }
  ]
}
```

Tạo file mới `lopC.json` , dán nội dung:

```
{
  "students": [
    {
      "id": 1,
```

```

        "name": "tuan"
      },
      {
        "id": 2,
        "name": "vu"
      }
    ]
  }
}

```

Bước 3: Viết code xử lý (🔪 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```

import fs from 'node:fs/promises'

console.log("Đọc file ...");

try {
  // Đọc CÙNG LÚC 3 file JSON bằng Promise.all (xem giải thích ở Bài 2.2-fs).
  // Mỗi biến nhận được là STRING (chuỗi JSON), chưa phải object.
  const [dulieuLopA, dulieuLopB, dulieuLopC] = await Promise.all([
    fs.readFile('./lopA.json', 'utf-8'),
    fs.readFile('./lopB.json', 'utf-8'),
    fs.readFile('./lopC.json', 'utf-8'),
  ]);

  // JSON.parse(chuoi) chuyển chuỗi JSON → object JavaScript thực sự.
  // Sau bước này mới truy cập được .students, .id, .name...
  const sinhvienLopA = JSON.parse(dulieuLopA);
  const sinhvienLopB = JSON.parse(dulieuLopB);
  const sinhvienLopC = JSON.parse(dulieuLopC);

  // Template literal `...${biểu thức}...` cho phép chèn JavaScript vào chuỗi.
  // .students.length đếm số phần tử trong mảng students của mỗi lớp, rồi cộng
  // lại.
  const ketqua = `Có tất cả ${sinhvienLopA.students.length +
    sinhvienLopB.students.length + sinhvienLopC.students.length} sinh viên`

  // fs.writeFile(tenFile, noiDung, encoding) GHI ĐÈ file nếu đã tồn tại, tạo mới
  // nếu chưa có.
  // Muốn THÊM vào cuối (không ghi đè): dùng fs.appendFile() thay thế.
  await fs.writeFile("baocao.txt", ketqua, "utf-8");
  console.log("Đã lưu kết quả vào file baocao.txt");
} catch (error) {
  // try/catch bắt lỗi: file không tồn tại, JSON sai cú pháp, không có quyền
  // ghi...
  console.log("Lỗi: ", error.message);
}

```

Giải thích:

1. **Đọc 3 file JSON song song** bằng `Promise.all` - cả 3 file được đọc cùng lúc, kết quả trả về là mảng 3 chuỗi.
2. **`JSON.parse()`** - Chuyển chuỗi JSON (text) thành object JavaScript. Sau bước này có thể truy cập `sinhvienLopA.students` như một mảng bình thường.
3. **Tính tổng** - Dùng `.length` để đếm số phần tử trong mảng `students` của mỗi lớp, rồi cộng lại: $4 + 3 + 2 = 9$.
4. **Template literal** - Dùng backtick ``` và `${}` để chèn biểu thức JavaScript vào chuỗi.
5. **`fs.writeFile()`** - Ghi nội dung vào file `baocao.txt`. Nếu file chưa tồn tại sẽ tự tạo mới, nếu đã tồn tại sẽ ghi đè.

Bước 4: Chạy chương trình (🖥️ TERMINAL)

```
node index.js
```

Kết quả trên Terminal:

```
Đọc file ...  
Đã lưu kết quả vào file baocao.txt
```

Kiểm tra file `baocao.txt` được tạo ra (🖥️ TERMINAL):

```
cat baocao.txt          # macOS / Linux  
# Hoặc trên Windows PowerShell:  
# Get-Content baocao.txt
```

Nội dung file:

```
Có tất cả 9 sinh viên
```

Hoặc bạn có thể mở file `baocao.txt` trực tiếp trong VS Code để xem.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
<code>SyntaxError: Unexpected token } in JSON at position...</code>	File JSON sai cú pháp (thừa dấu phẩy, thiếu dấu ngoặc...)	Kiểm tra lại file JSON. Mỗi key và value chuỗi phải dùng dấu ngoặc kép ". Không được có dấu phẩy sau phần tử cuối cùng
<code>TypeError: Cannot read properties of undefined (reading 'length')</code>	Tên property sai (vd: <code>student</code> thay vì <code>students</code>)	Kiểm tra tên property trong file JSON và trong code phải khớp nhau
<code>ENOENT: no such file or directory, open './lopA.json'</code>	File JSON không tồn tại hoặc sai đường dẫn	Đảm bảo các file <code>.json</code> nằm cùng thư mục với <code>index.js</code>
File <code>baocao.txt</code> bị ghi đè mỗi lần chạy	<code>writeFile</code> mặc định ghi đè toàn bộ	Đây là hành vi bình thường. Nếu muốn ghi thêm (append), dùng <code>fs.appendFile()</code> thay vì <code>fs.writeFile()</code>

Chương 3: Express.js Cơ Bản

Bài 3.1: Node Modules & NPM

Mục tiêu

- Hiểu cách quản lý package với NPM
- Cài đặt và sử dụng package bên ngoài (`express` , `nodemon`)
- Sử dụng `nodemon` để tự động restart server khi thay đổi code

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 3.1.node-modules
cd 3.1.node-modules
npm init -y
```

Bước 1b: Bật ES Modules (📝 SỬA FILE)

Thêm `"type": "module"` vào `package.json` .

Bước 2: Cài đặt Express và Nodemon (🖥️ TERMINAL)

```
# Cài Express vào dependencies (cần khi chạy production)
npm install express
# Cài Nodemon vào devDependencies (chỉ cần khi phát triển)
npm install nodemon --save-dev
```

Giải thích:

- `npm install express` - Cài Express vào `dependencies` (cần cho production).
- `npm install nodemon --save-dev` - Cài Nodemon vào `devDependencies` (chỉ cần khi phát triển). Nodemon tự động restart server mỗi khi bạn lưu file, không cần `Ctrl+C` rồi `node index.js` lại.

Sau khi cài, thư mục `node_modules/` và file `package-lock.json` sẽ được tạo tự động.

Bước 3: Thêm script chạy với Nodemon (📝 SỬA FILE)

Mở `package.json` , thêm script `"dev"` vào phần `"scripts"` :

```
{
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "dev": "nodemon index.js"
  }
}
```

Bước 4: Viết code (📝 TẠO FILE)

Tạo file mới `index.js` , dán nội dung:

```
// File này dùng module http thuần (giống Bài 1.4) – chưa dùng Express vôi.  
// Mục đích của Bài 3.1 là làm quen với npm + nodemon trước.  
import http from 'node:http'  
  
const server = http.createServer((req, res) => {  
  res.end('Hello world!');  
})  
  
server.listen(3000, () => {  
  console.log('Server is running ...');  
})
```

Bước 5: Tạo file .gitignore (📝 TẠO FILE)

Tạo file mới `.gitignore` ở thư mục gốc project, dán nội dung:

```
node_modules
```

Tại sao cần `.gitignore` ? Thư mục `node_modules/` rất lớn (hàng ngàn file). Không nên đưa lên Git. Khi cần cài lại, chỉ cần chạy `npm install` - NPM sẽ đọc `package.json` và tải lại tất cả.

Bước 6: Chạy server (💻 TERMINAL)

Cách 1 - Chạy thường:

```
node index.js
```

Cách 2 - Chạy với Nodemon (khuyến dùng khi phát triển):

```
npm run dev
```

Khi dùng Nodemon, mỗi khi bạn sửa và lưu file `index.js`, server sẽ tự restart.

Kết quả trên Terminal:

```
Server is running ...
```

Test (🌐 TRÌNH DUYỆT): Truy cập `http://localhost:3000` → thấy `Hello world!`

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: Cannot find module 'express'	Chưa cài express	Chạy <code>npm install express</code>
'nodemon' is not recognized as a command	Chưa cài nodemon hoặc chưa dùng script	Chạy <code>npm install nodemon --save-dev</code> rồi dùng <code>npm run dev</code> thay vì <code>nodemon index.js</code> trực tiếp

Thư mục node_modules quá lớn	Đây là bình thường	Thêm node_modules vào .gitignore, không đưa lên Git
------------------------------	--------------------	---

Bài 3.3: Express, Morgan & Các Loại Tham Số Request

Mục tiêu

- Tạo ứng dụng Express.js đầu tiên
- Sử dụng middleware Morgan để ghi log request
- Phân biệt 3 loại tham số: **Query Params, Route Params, Request Body**
- Gửi request POST với custom header

Bước 1: Tạo project và cài đặt (🖥️ TERMINAL)

```
mkdir 3.3-morgan
cd 3.3-morgan
npm init -y
npm install express morgan
```

Bước 1b: Bật ES Modules (📝 SỬA FILE)

Thêm `"type": "module"` vào `package.json`.

Bước 2: Viết code (📝 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
// Import package express (đã cài qua npm install).
import express from 'express';
// Import package morgan - middleware ghi log mỗi request vào terminal.
import morgan from 'morgan';

// Tạo ứng dụng Express - app này có sẵn các phương thức .get(), .post(), .use()...
const app = express();

// app.use() = ĐĂNG KÝ MIDDLEWARE chạy cho MỌI request.
// express.json() = middleware phân tích body JSON từ request → gắn vào req.body.
// Nếu thiếu dòng này, req.body sẽ là undefined khi gửi JSON.
app.use(express.json())

// morgan('common') = log mỗi request theo format "common" (IP, time, method, URL, status...).
app.use(morgan('common'))

// params (các loại tham số)

// 1. QUERY PARAMS - tham số nằm sau dấu ? trên URL.
// Ví dụ URL: http://localhost:3000?search=test → req.query = { search: 'test' }
app.get('/', (req, res) => {
  res.end('Hello ' + req.query.search);
});
```

```

})

// 2. ROUTE PARAMS - tham số nằm TRONG đường dẫn, đánh dấu bằng dấu ':'.
// Ví dụ URL: http://localhost:3000/test → req.params = { search: 'test' }
// Có thể kết hợp với query params: /test?search=abc
app.get('/:search', (req, res) => {
  res.end('Hello ' + req.params.search + ' with name: ' + req.query.search);
})

// REQUEST BODY - dữ liệu client gửi kèm (thường với POST/PUT).
app.post('/', (req, res) => {
  // req.headers chứa tất cả header. Tên header viết thường (HTTP không phân biệt
  // hoa/thường).
  const header_info = req.headers['my-header']
  console.log('my-header: ', header_info);

  // res.json(data) tự động: 1) Set header Content-Type: application/json
  //                               2) Chuyển data thành chuỗi JSON 3) Gửi về client.
  res.json(req.body.university)
})

app.listen(3000, () => {
  console.log('Server is running ...');
})

```

Giải thích:

- **express()** - Tạo ứng dụng Express.
- **app.use(express.json())** - Middleware giúp Express đọc được dữ liệu JSON từ body request.
- **app.use(morgan('common'))** - Middleware ghi log mỗi request vào Terminal (IP, thời gian, method, URL, status code).
- **Query Params** (req.query) - Tham số trên URL sau dấu **?**. VD: **?search=test** -> req.query.search = "test".
- **Route Params** (req.params) - Tham số nằm trong đường dẫn URL. VD: **/:search** khi truy cập **/hello** -> req.params.search = "hello".
- **Request Body** (req.body) - Dữ liệu gửi kèm trong body (thường dùng với POST/PUT).
- **Request Headers** (req.headers) - Thông tin header của request.

Bước 3: Chạy server (🖥️ TERMINAL)

```
node index.js
```

Bước 4: Test các loại tham số

Test 1 - Query Params (🌐 TRÌNH DUYỆT): Mở trình duyệt, truy cập:

```
http://localhost:3000?search=nodejs
```

Kết quả hiển thị: Hello nodejs

Test 2 - Route Params (🌐 TRÌNH DUYỆT): Mở trình duyệt:

```
http://localhost:3000/express?search=framework
```

Kết quả: `Hello express with name: framework`

Test 3 - POST request (🖥️ TERMINAL MỚI): Mở Terminal mới (giữ Terminal cũ chạy server), dùng curl (hoặc Postman, Thunder Client):

```
curl -X POST http://localhost:3000 \
-H "Content-Type: application/json" \
-H "my-header: xin-chao" \
-d '{"university": "Dong A"}
```

Kết quả trả về: `"Dong A"`

Quan sát Terminal đang chạy server: sẽ thấy log Morgan như sau:

```
:::1 - - [12/May/2025:10:30:00 +0000] "GET /?search=nodejs HTTP/1.1" 200 12
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
req.body là undefined	Thiếu app.use(express.json())	Thêm app.use(express.json()) trước các route
req.query.search là undefined	URL thiếu query param	Đảm bảo URL có dạng ?search=giaTri
Cannot GET /favicon.ico	Trình duyệt tự request icon	Bỏ qua, đây là hành vi bình thường của trình duyệt

Bài 3.th: Thực Hành - Xây Dựng RESTful API CRUD Sinh Viên

Mục tiêu

- Xây dựng API RESTful hoàn chỉnh với 5 endpoint CRUD
- Sử dụng đúng HTTP method cho từng thao tác
- Kết hợp route params và request body

Bước 1: Tạo project và cài đặt (🖥️ TERMINAL)

```
mkdir 3.th
cd 3.th
npm init -y
npm install express morgan
npm install nodemon --save-dev
```

Bước 1b: Cấu hình package.json (📄 SỬA FILE)

Mở package.json , thêm "type": "module" và script "dev" :

```
{
  "type": "module",
  "scripts": {
    "dev": "nodemon index.js"
  }
}
```

Bước 2: Viết code (👉 TẠO FILE)

Tạo file mới `index.js` , dán nội dung:

```
import express from 'express';
import morgan from 'morgan';

const app = express();
app.use(express.json())

// morgan('dev') = format log ngắn, có MÀU SẮC theo status code (xanh = OK, đỏ = lỗi).
app.use(morgan('dev'))

// GET /sinhvien/ → LẤY DANH SÁCH tất cả sinh viên (Read all – R trong CRUD)
app.get('/sinhvien/', (req, res) => {
  // res.send() tự nhận biết kiểu dữ liệu (string, object, buffer) để set Content-Type đúng.
  res.send('Danh sách sinh viên là: ...')
})

// POST /sinhvien/ → TẠO MỚI sinh viên (Create – C trong CRUD)
// Dữ liệu sinh viên mới được gửi trong body request dạng JSON.
app.post('/sinhvien/', (req, res) => {
  res.json(req.body); // Trả ngược lại body để client xác nhận đã nhận đúng
})

// GET /sinhvien/:id → LẤY 1 sinh viên theo mã (Read one)
// :id là route param – lấy bằng req.params.id
app.get('/sinhvien/:id', (req, res) => {
  res.send('Thông tin sinh viên có mã ' + req.params.id)
})

// PUT /sinhvien/:id → CẬP NHẬT sinh viên có id tương ứng (Update – U)
app.put('/sinhvien/:id', (req, res) => {
  // JSON.stringify(object) chuyển object → chuỗi JSON để nối vào chuỗi text.
  res.send('Đã sửa thông tin sinh viên có mã ' + req.params.id + ' với chi tiết: ' + JSON.stringify(req.body))
})

// DELETE /sinhvien/:id → XOÁ sinh viên (Delete – D)
app.delete('/sinhvien/:id', (req, res) => {
  // Trong thực tế cần:
  // 1) Kiểm tra sinh viên :id có tồn tại không (truy vấn DB)
```

```
// 2) Nếu có → xóa; nếu không → trả status 404
// Hiện tại đang là demo nên chỉ trả thông báo.
res.send('Đã xóa sinh viên ' + req.params.id)
})

app.listen(3000, () => {
  console.log('server is running ...');
})
```

Giải thích:

Đây là mô hình **RESTful API** - sử dụng HTTP method phù hợp với từng thao tác:

HTTP Method	Endpoint	Thao tác	Ý nghĩa
GET	/sinhvien/	Read all	Lấy danh sách tất cả sinh viên
POST	/sinhvien/	Create	Tạo sinh viên mới
GET	/sinhvien/:id	Read one	Lấy thông tin 1 sinh viên theo mã
PUT	/sinhvien/:id	Update	Cập nhật thông tin sinh viên
DELETE	/sinhvien/:id	Delete	Xoá sinh viên

- `morgan('dev')` - Format log ngắn gọn, có màu sắc: `GET /sinhvien/ 200 3.456 ms`
- `res.send()` - Gửi text response.
- `res.json()` - Gửi JSON response.
- `JSON.stringify(req.body)` - Chuyển object thành chuỗi JSON để hiển thị.

Bước 3: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 4: Test bằng curl (🖥️ TERMINAL MỚI)

Mở Terminal MỚI (giữ Terminal cũ đang chạy server), chạy từng lệnh bên dưới để test 5 thao tác CRUD:

```
# Lấy danh sách sinh viên (GET)
curl http://localhost:3000/sinhvien/

# Tạo sinh viên mới (POST)
curl -X POST http://localhost:3000/sinhvien/ \
  -H "Content-Type: application/json" \
  -d '{"name": "Nguyen Van A", "lop": "20A"}'

# Xem sinh viên có mã 1 (GET)
curl http://localhost:3000/sinhvien/1

# Sửa sinh viên có mã 1 (PUT)
curl -X PUT http://localhost:3000/sinhvien/1 \
  -H "Content-Type: application/json" \
```

```
-d '{"name": "Nguyen Van B"}'
```

```
# Xoá sinh viên có mã 1 (DELETE)
```

```
curl -X DELETE http://localhost:3000/sinhvien/1
```

Quan sát Terminal chạy server: sẽ thấy log Morgan (format `dev`):

```
GET /sinhvien/ 200 1.234 ms - 30
POST /sinhvien/ 200 0.567 ms - 42
GET /sinhvien/1 200 0.345 ms - 35
PUT /sinhvien/1 200 0.432 ms - 58
DELETE /sinhvien/1 200 0.234 ms - 20
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Cannot GET /sinhvien (không có / cuối)	Đường dẫn không khớp	Thử cả /sinhvien và /sinhvien/, hoặc bỏ / cuối trong route
POST trả về {} rỗng	Thiếu Content-Type: application/json trong header	Thêm header -H "Content-Type: application/json" khi gửi request
req.params.id luôn là string	Route params luôn trả về string	Dùng Number(req.params.id) hoặc parseInt(req.params.id) nếu cần so sánh số

Chương 4: Kiến Trúc MVC

Bài 4.0: So Sánh HTTP Thuần vs Express

Mục tiêu

- Hiểu tại sao cần Express.js
- So sánh cách tạo server và xử lý route giữa module `http` thuần và Express

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 4.0.no-express
cd 4.0.no-express
npm init -y
npm install express
```

Bước 1b: Bật ES Modules (📝 SỬA FILE)

Thêm `"type": "module"` vào `package.json`.

Bước 2: Tạo server KHÔNG dùng Express (📝 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
import http from 'http'

// Khi DÙNG http thuần (không Express), MỌI request đều đi vào CALLBACK NÀY.
// Bạn phải tự kiểm tra req.url, req.method để biết client muốn gì.
const server = http.createServer((req, res) => {
  // req.url chứa phần URL phía sau domain (vd: "/sinhvien", "/users/123")
  if (req.url === '/sinhvien') {
    res.end('Hello from Sinhvien!');
  }
  else if (req.url === '/') {
    res.end('Hello world!');
  }
  // Nếu URL khác → KHÔNG có nhánh nào chạy → request bị treo!
  // (Điểm yếu của http thuần: phải xử lý mọi trường hợp thủ công)
})

server.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 3: Tạo server CÓ dùng Express (📝 TẠO FILE)

Tạo file mới `index-express.js` (cùng cấp với `index.js`), dán nội dung:

```
import express from 'express'
```

```
const app = express();

// Với Express: mỗi route được khai báo RIÊNG bằng app.get(), app.post()...
// Express tự match URL → tự gọi handler đúng → code GỌN HƠN nhiều so với if/else.
app.get('/', (req, res) => {
  res.send('Hello world from express!')
})
app.get('/sinhvien', (req, res) => {
  res.send('Hello from Sinh vien Express!')
})

// Nếu URL không khớp route nào, Express tự trả về "Cannot GET /xxx" (status 404).

app.listen(3000, () => {
  console.log('Server is running with express ...');
})
```

Bước 4: Chạy và so sánh (🖥️ TERMINAL)

```
# Chạy bản HTTP thuần
node index.js

# Hoặc chạy bản Express (dừng server trước bằng Ctrl+C)
node index-express.js
```

Test (🌐 TRÌNH DUYỆT): Truy cập `http://localhost:3000` và `http://localhost:3000/sinhvien` để xem kết quả của từng phiên bản.

So sánh:

	HTTP thuần (http)	Express
Routing	Dùng if/else if kiểm tra req.url	Dùng app.get(), app.post()... rõ ràng
Response	res.end()	res.send(), res.json() - nhiều tiện ích
Middleware	Phải tự viết	Có sẵn hệ thống middleware
Khi nhiều route	Code rất dài, khó quản lý	Mỗi route 1 block, dễ đọc

Express giúp code ngắn gọn, dễ đọc và dễ mở rộng hơn rất nhiều so với dùng module `http` trực tiếp.

Bài 4.4-mvc: Giới Thiệu Kiến Trúc MVC

Mục tiêu

- Hiểu mô hình **MVC (Model - View - Controller)**
- Tách code thành các tầng: Model, Controller, Route
- Sử dụng **Express Router** để quản lý route

Kiến thức MVC

Request → Route → Controller → Model → Response

Tầng	Vai trò	Ví dụ
Model	Quản lý dữ liệu	Mảng users, kết nối database
Controller	Xử lý logic nghiệp vụ	Lấy dữ liệu, tính toán, trả response
Route	Định nghĩa endpoint & HTTP method	GET /users, POST /users
View	Giao diện (sẽ học ở Chương 5)	Template HTML

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 4.4-mvc
cd 4.4-mvc
npm init -y
npm install express
npm install nodemon --save-dev
```

Bước 1b: Cấu hình package.json (📄 SỬA FILE)

Thêm "type": "module" và script "dev": "nodemon app.js" vào package.json .

Bước 1c: Tạo cấu trúc thư mục (🖥️ TERMINAL)

```
mkdir models controllers routes views
```

Cấu trúc project bạn sẽ có:

```
4.4-mvc/
├─ app.js
├─ models/
│   └─ user.models.js
├─ controllers/
│   └─ user.controller.js
├─ routes/
│   └─ user.route.js
└─ views/    (trống – sẽ dùng ở chương sau)
```

Bước 2: Tạo Model (📄 TẠO FILE)

Tạo file mới models/user.models.js , dán nội dung:

```
// 'let' (thay vì 'const') vì sau này mảng có thể bị thay đổi (thêm/xoá/sửa phần tử).
// Mỗi user là một object có id (số) và name (chuỗi).
let users = [
  {
    id: 1,
```

```

    name: "tuan"
  },
  {
    id: 2,
    name: "vu"
  }
]

// export default = xuất ra GIÁ TRỊ MẶC ĐỊNH của file.
// Khi import: import users from './user.models.js' (đặt tên gì cũng được).
export default users

```

Giải thích: Model chứa dữ liệu. Hiện tại dùng mảng trong bộ nhớ, sau này sẽ thay bằng database.

Bước 3: Tạo Controller (📝 TẠO FILE)

Tạo file mới `controllers/user.controller.js`, dán nội dung:

```

// Import mảng users từ Model.
// Đường dẫn '../models/...' = lùi 1 cấp (ra khỏi 'controllers/') rồi vào 'models/'.
// LƯU Ý: ES Module YÊU CẦU đuôi .js đầy đủ (không được bỏ).
import users from '../models/user.models.js'

// export const = named export. Khi import: import { getAll } from '...' (đúng tên).
// getAll = handler cho route "lấy tất cả users".
export const getAll = (req, res) => {
  res.json(users) // Gửi mảng users dạng JSON về client
}

// create = handler cho route "tạo user mới".
export const create = (req, res) => {
  // Tạo object user mới:
  // id = số lượng phần tử hiện tại + 1 (cách tạm thời, không an toàn khi xóa
  // giữa chừng)
  // name = lấy từ req.body.name (client gửi lên)
  const user = {
    id: users.length + 1,
    name: req.body.name
  }
  // .push(item) thêm phần tử vào CUỐI mảng. Vì 'users' chỉ là biến trong RAM →
  // mất khi restart.
  users.push(user)
  res.json(user) // Trả về user vừa tạo để client biết kết quả
}

```

Giải thích: Controller chứa logic xử lý. Mỗi hàm nhận `req`, `res` và thực hiện một thao tác cụ thể. `export const` cho phép export từng hàm riêng lẻ (named export).

Bước 4: Tạo Route (📝 TẠO FILE)

Tạo file mới `routes/user.route.js`, dán nội dung:

```
// Router là class của Express dùng để tạo các "mini-app" có route riêng.
// { Router } = named import (lấy đúng cái tên Router từ express).
import { Router } from "express";
// Import 2 hàm controller bằng destructuring (named import).
import { getAll, create } from "../controllers/user.controller.js";

// Tạo instance Router mới.
const router = Router()

// Khai báo route TRÊN router (không phải trên app).
// Đường dẫn '' (rỗng) vì prefix '/users' sẽ được gắn ở app.js.
// → router.get('', ...) ứng với GET /users (sau khi prefix).
router.get('', getAll)
router.post('', create)

// Export router để app.js có thể import và gắn vào app.
export default router
```

Giải thích: Router() tạo một mini-router riêng. Các route ở đây dùng đường dẫn '' (rỗng) vì prefix /users sẽ được gắn trong app.js .

Bước 5: Tạo file chính app.js (📁 TẠO FILE)

Tạo file mới app.js ở thư mục gốc project (cùng cấp với package.json), dán nội dung:

```
import express from 'express'
// Import router đã tạo (default import → đặt tên gì cũng được, ở đây là
userRouter).
import userRouter from './routes/user.route.js'

const app = express()
app.use(express.json()) // Bật phân tích JSON body

// app.use(prefix, router) = "gắn" router vào prefix.
// Mọi route trong userRouter sẽ tự động được thêm '/users' phía trước.
// → router.get('') trở thành GET /users
// → router.post('') trở thành POST /users
app.use('/users', userRouter)

app.listen(3000, (req, res) => {
  console.log('Server is running ...');
})
```

Giải thích: app.use('/users', userRouter) - Gắn toàn bộ userRouter vào prefix /users . Nghĩa là:

- router.get('') -> GET /users
- router.post('') -> POST /users

Bước 6: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 7: Test API (🖥️ TERMINAL MỚI)

Mở Terminal MỚI (giữ Terminal cũ đang chạy server).

Test 1 - Lấy danh sách users:

```
curl http://localhost:3000/users
```

Kết quả:

```
[{"id":1,"name":"tuan"}, {"id":2,"name":"vu"}]
```

Test 2 - Tạo user mới:

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name": "nguyen"}'
```

Kết quả:

```
{"id":3,"name":"nguyen"}
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ERR_MODULE_NOT_FOUND khi import	Thiếu đuôi .js trong đường dẫn import	ES Module yêu cầu ghi đầy đủ: './routes/user.route.js' (không được bỏ .js)
TypeError: Router is not a function	Import sai cú pháp	Dùng import { Router } from "express" (destructuring)
Cannot GET /users/ (có / cuối)	Route định nghĩa không có / cuối	Thử bỏ / cuối trong URL hoặc thêm / trong route

Bài 4.4.thuchanh: Thực Hành - API Users Đơn Giản (Chứa MVC)

Mục tiêu

- Viết API quản lý users đơn giản trong 1 file
- Hiểu tại sao cần tách ra MVC khi code lớn dần

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 4.4.thuchanh
cd 4.4.thuchanh
npm init -y
```

```
npm install express
npm install nodemon --save-dev
```

Bước 1b: Cấu hình `package.json` (✏️ SỬA FILE)

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Viết code - tất cả trong 1 file (✏️ TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
import express from 'express'

const app = express()
app.use(express.json())

// MODEL (dữ liệu) - viết thẳng trong file này (chưa tách ra theo MVC).
let users = [
  {
    id: 1,
    name: "tuan"
  },
  {
    id: 2,
    name: "vu"
  }
]

// ROUTE + CONTROLLER (logic xử lý) - đều viết trong file này.
// Lấy danh sách users
app.get('/users', (req, res) => {
  res.json(users)
})

// Tạo user mới
app.post('/users', (req, res) => {
  const user = {
    id: users.length + 1,
    name: req.body.name
  }
  users.push(user)
  res.json(user)
})

// → Khi project có nhiều resource (users, products, orders...) file này sẽ rất dài.
// Đó là lý do cần tách MVC như Bài 4.4-mvc.
app.listen(3000, () => {
  console.log('Server is running');
})
```

Bước 3: Chạy và test (💻 TERMINAL)

```
npm run dev
```

Test giống bài 4.4-mvc ở trên (Bước 7 - dùng curl trong Terminal mới). Kết quả giống nhau, nhưng **tất cả code nằm trong 1 file**.

Nhận xét: Bài này làm được nhưng khi project lớn (nhiều resource: users, products, orders...), file sẽ rất dài và khó quản lý. Đó là lý do cần tách ra theo MVC như bài 4.4-mvc.

Bài 4.4.thuchanh-2: Thực Hành - CRUD Đầy Đủ Với MVC (Nhiều Resource)

Mục tiêu

- Áp dụng MVC cho nhiều resource (customers & products)
- Viết đầy đủ 5 thao tác CRUD: getAll, getOne, create, update, delete
- Xử lý lỗi với HTTP status code phù hợp (404, 204)

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 4.4.thuchanh-2
cd 4.4.thuchanh-2
npm init -y
npm install express
npm install nodemon --save-dev
```

Bước 1b: Cấu hình package.json (✏️ SỬA FILE)

Thêm "type": "module" và script "dev": "nodemon index.js" vào package.json .

Bước 1c: Tạo cấu trúc thư mục (🖥️ TERMINAL)

```
mkdir models controllers routes
```

Cấu trúc project bạn sẽ có:

```
4.4.thuchanh-2/
├── index.js
├── models/
│   ├── customer.model.js
│   └── product.model.js
├── controllers/
│   ├── customer.controller.js
│   └── product.controller.js
└── routes/
    ├── customer.route.js
    └── product.route.js
```

Bước 2: Tạo Models (✏️ TẠO FILE)

Tạo file mới `models/customer.model.js` , dán nội dung:

```
// Model customers - mảng dữ liệu mẫu cho thao tác CRUD.
let customers = [
  {
    id: 1,
    name: "tuan"
  },
  {
    id: 2,
    name: "vu"
  },
  {
    id: 3,
    name: "nguyen"
  }
]

export default customers
```

Tạo file mới `models/product.model.js` , dán nội dung:

```
let products = [
  {
    id: 1,
    name: "Laptop",
    price: 1000
  },
  {
    id: 2,
    name: "Phone",
    price: 200
  }
]

export default products
```

Bước 3: Tạo Controllers - CRUD đầy đủ (📝 TẠO FILE)

Tạo file mới `controllers/customer.controller.js` , dán nội dung:

```
import customers from "../models/customer.model.js"

// READ ALL: Lấy tất cả customers
export const getAll = (req, res) => {
  res.json(customers)
}

// CREATE: Tạo customer mới
export const create = (req, res) => {
```

```

    const newCustomer = {
      id: customers.length + 1,
      // ...req.body = SPREAD OPERATOR: sao chép TẤT CẢ property từ req.body sang
      // object mới.
      // Ví dụ req.body = {name:"a", age:20} → kết quả {id:4, name:"a", age:20}
      ...req.body
    }
    customers.push(newCustomer)
    res.json(newCustomer)
  }

// READ ONE: Lấy 1 customer theo id
export const getOne = (req, res) => {
  // .find(callback) duyệt mảng, trả về PHẦN TỬ ĐẦU TIÊN thỏa điều kiện, hoặc
  // undefined nếu không có.
  // Number(req.params.id) - chuyển string "1" → number 1 vì req.params luôn là
  // string.
  const customer = customers.find(c => c.id === Number(req.params.id))
  if (!customer) {
    // res.status(404) set HTTP status. .json() gửi body và kết thúc response.
    // return để dừng hàm tại đây (tránh chạy tiếp res.json bên dưới gây lỗi).
    return res.status(404).json({ error: 'Not found' })
  }
  res.json(customer)
}

// UPDATE: Cập nhật customer
export const update = (req, res) => {
  // .findIndex() trả về VỊ TRÍ (chỉ số) trong mảng, hoặc -1 nếu không tìm thấy.
  const index = customers.findIndex(c => c.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  // Ghi đè phần tử tại vị trí index bằng object MỚI:
  // - ...customers[index] = copy các field cũ
  // - ...req.body = override bằng dữ liệu mới (chỉ field nào có trong
  // body mới ghi đè)
  // Kết quả: chỉ field được gửi mới bị thay, các field khác giữ nguyên.
  customers[index] = {
    ...customers[index],
    ...req.body
  }
  res.json(customers[index])
}

// DELETE: Xóa customer
export const remove = (req, res) => {
  const index = customers.findIndex(c => c.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  // .splice(start, deleteCount) - xóa 1 phần tử tại vị trí index.

```

```

    customers.splice(index, 1)
    // Status 204 = "No Content" (thành công nhưng không có dữ liệu trả về).
    // .end() đóng response mà không gửi body.
    res.status(204).end()
  }
}

```

Giải thích các kỹ thuật quan trọng:

- **...req.body** (Spread operator) - Sao chép tất cả property từ body vào object mới, không cần liệt kê từng field.
- **Number(req.params.id)** - Chuyển string thành number vì **req.params** luôn trả về string.
- **.find()** - Tìm phần tử đầu tiên khớp điều kiện, trả về **undefined** nếu không tìm thấy.
- **.findIndex()** - Tìm vị trí (index) của phần tử, trả về **-1** nếu không tìm thấy.
- **.splice(index, 1)** - Xóa 1 phần tử tại vị trí **index**.
- **res.status(404)** - Trả về HTTP status 404 (Not Found).
- **res.status(204).end()** - Trả về HTTP status 204 (No Content) - thành công nhưng không có nội dung trả về (thường dùng cho DELETE).

Tạo file mới **controllers/product.controller.js** (logic tương tự, thay **customers** bằng **products**):

```

// Controller cho Product - logic tương tự customer.controller.js, chỉ đổi biến.
// Xem giải thích chi tiết ở customer.controller.js bên trên.
import products from "../models/product.model.js";

export const getAll = (req, res) => {
  res.json(products)
}

export const create = (req, res) => {
  const newProduct = {
    id: products.length + 1,
    ...req.body // Sao chép name, price... từ body request
  }
  products.push(newProduct)
  res.json(newProduct)
}

export const getOne = (req, res) => {
  const product = products.find(p => p.id === Number(req.params.id))
  if (!product) {
    return res.status(404).json({ error: 'Not found' })
  }
  res.json(product)
}

export const update = (req, res) => {
  const index = products.findIndex(p => p.id === Number(req.params.id))
  if (index === -1) {
    return res.status(404).json({ error: 'Not found' })
  }
  products[index] = {
    ...products[index], // Giữ các field cũ

```

```

        ...req.body // Ghi đè bằng field mới (chỉ field nào client gửi)
    }
    res.json(products[index])
}

export const remove = (req, res) => {
    const index = products.findIndex(p => p.id === Number(req.params.id))
    if (index === -1) {
        return res.status(404).json({ error: 'Not found' })
    }
    products.splice(index, 1)
    res.status(204).end()
}

```

Bước 4: Tạo Routes (📝 TẠO FILE)

Tạo file mới `routes/customer.route.js`, dán nội dung:

```

import { Router } from "express";
// Import 5 hàm controller cùng lúc (named imports).
import { create, getAll, getOne, remove, update } from
"../controllers/customer.controller.js";

// new Router() và Router() đều OK (Router là hàm constructor, có thể gọi 2 cách).
const router = new Router();

// Đăng ký 5 route ứng với 5 thao tác CRUD.
// LƯU Ý THỨ TỰ: route cụ thể('/:id' phải đặt SAU route '/' để Express khớp đúng.
router.get('/', getAll) // GET /customers → lấy tất cả
router.get('/:id', getOne) // GET /customers/:id → lấy 1
router.put('/:id', update) // PUT /customers/:id → cập nhật
router.delete('/:id', remove) // DELETE /customers/:id → xoá
router.post('/', create) // POST /customers → tạo mới

export default router;

```

Tạo file mới `routes/product.route.js`, dán nội dung:

```

import { Router } from "express";
import { create, getAll, getOne, remove, update } from
"../controllers/product.controller.js";

const router = Router()

router.get('/', getAll)
router.get('/:id', getOne)
router.put('/:id', update)
router.delete('/:id', remove)
router.post('/', create)

```

```
export default router
```

Bước 5: Tạo file chính (👉 TẠO FILE)

Tạo file mới `index.js` ở thư mục gốc project, dán nội dung:

```
import express from 'express'
// Import 2 router (mỗi resource 1 router).
import productRouter from './routes/product.route.js'
import customerRouter from './routes/customer.route.js'

const app = express();
app.use(express.json())

// Gắn router vào prefix tương ứng.
// Cách tổ chức này giúp dễ thêm resource mới: chỉ cần thêm 1 dòng app.use(...).
app.use('/products', productRouter)
app.use('/customers', customerRouter)

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 6: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 7: Test API (🖥️ TERMINAL MỚI)

Mở Terminal MỚI để test (giữ Terminal cũ đang chạy server).

Test Customer API:

```
# Lấy tất cả customers
curl http://localhost:3000/customers

# Lấy customer có id = 1
curl http://localhost:3000/customers/1

# Tạo customer mới
curl -X POST http://localhost:3000/customers \
  -H "Content-Type: application/json" \
  -d '{"name": "tran"}'

# Cập nhật customer có id = 1
curl -X PUT http://localhost:3000/customers/1 \
  -H "Content-Type: application/json" \
  -d '{"name": "tuan updated"}'
```

```
# Xoá customer có id = 2
curl -X DELETE http://localhost:3000/customers/2
```

Test Product API:

```
# Lấy tất cả products
curl http://localhost:3000/products

# Tạo product mới
curl -X POST http://localhost:3000/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Tablet", "price": 500}'

# Lấy product không tồn tại
curl http://localhost:3000/products/99
```

Kết quả khi product không tồn tại (status 404):

```
{"error": "Not found"}
```

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
ERR_MODULE_NOT_FOUND	Thiếu .js trong import path	Luôn ghi đầy đủ đuôi .js: '../models/customer.model.js'
GET /customers/abc trả về 404	Number("abc") = NaN, không khớp id nào	Đây là đúng hành vi. Có thể thêm validation nếu muốn thông báo lỗi rõ hơn
DELETE trả về rỗng	Status 204 nghĩa là No Content	Đúng hành vi. Kiểm tra status code = 204 là thành công
Dữ liệu mất khi restart server	Dữ liệu lưu trong bộ nhớ (mảng)	Đây là bình thường. Sẽ học lưu vào database ở các chương sau

Chương 5: Template Engine - EJS

Bài 5.2: Ứng Dụng Web Với EJS

Mục tiêu

- Sử dụng **EJS** (Embedded JavaScript) làm template engine để render HTML động
- Tích hợp EJS vào kiến trúc MVC
- Sử dụng **partials** (header, footer) để tái sử dụng giao diện
- Phục vụ file tĩnh (CSS, JS, hình ảnh) với `express.static`

Bước 1: Tạo project và cài đặt (🖥️ TERMINAL)

```
mkdir 5.2.ejs
cd 5.2.ejs
npm init -y
npm install express ejs
npm install nodemon --save-dev
```

Bước 1b: Cấu hình `package.json` (📄 SỬA FILE)

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Tạo cấu trúc thư mục (🖥️ TERMINAL)

```
# Flag -p cho phép tạo nhiều cấp thư mục cùng lúc.
# Trên Windows PowerShell có thể cần tạo từng cái riêng.
mkdir -p models controllers routes views/partial views/users public/css public/js
public/images
```

Cấu trúc project bạn sẽ có:

```
5.2.ejs/
├─ index.js
├─ models/
│  └─ user.model.js
├─ controllers/
│  └─ user.controller.js
├─ routes/
│  └─ user.route.js
├─ views/
│  ├─ partials/
│  │  ├─ header.ejs
│  │  └─ footer.ejs
│  └─ users/
│     ├─ index.ejs
│     └─ detail.ejs
└─ public/
   └─ css/
```

```
|   └─ style.css
└─ js/
    └─ main.js
└─ images/
    └─ logo.png
```

Bước 3: Tạo Model (📄 TẠO FILE)

Tạo file mới `models/user.model.js` , dán nội dung:

```
// Model users với thêm field 'age' để render đa dạng hơn trên view.
let users = [
  {
    id: 1,
    name: "tuan",
    age: 20
  },
  {
    id: 2,
    name: "vu",
    age: 21
  }
]

export default users
```

Bước 4: Tạo Controller (📄 TẠO FILE)

Tạo file mới `controllers/user.controller.js` , dán nội dung:

```
import users from "../models/user.model.js";

// res.render(tenView, dataObject) - khác với res.json/res.send:
// 1) Tìm file template trong thư mục views (vd: views/users/index.ejs)
// 2) Render template với data, kết quả ra HTML
// 3) Gửi HTML về cho trình duyệt
export const getAll = (req, res) => {
  // { users } = shorthand của { users: users } (ES6 object property).
  // Trong template index.ejs sẽ truy cập biến 'users' này.
  res.render('users/index', { users })
}

export const getOne = (req, res) => {
  // Tìm user theo id (Number để chuyển string param → number).
  const user = users.find(u => u.id === Number(req.params.id))
  // Render template detail.ejs, truyền user vào để hiển thị.
  res.render('users/detail', { user })
}
```

Giải thích:

- `res.render('users/index', { users })` - Thay vì trả JSON (`res.json`), bây giờ render template EJS. Tham số thứ 2 là object dữ liệu truyền vào template.
- `{ users }` là viết tắt ES6 của `{ users: users }` .

Bước 5: Tạo Route (📝 TẠO FILE)

Tạo file mới `routes/user.route.js` , dán nội dung:

```
import { Router } from 'express'
import { getAll, getOne } from '../controllers/user.controller.js'

const router = Router();

// 2 route hiển thị trang HTML (không phải JSON API).
router.get('/', getAll)      // GET /users      → render danh sách
router.get('/:id', getOne)   // GET /users/:id → render chi tiết 1 user

export default router
```

Bước 6: Tạo Views - Template EJS (📝 TẠO FILE)

Tạo file mới `views/partials/header.ejs` , dán nội dung:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <link rel="stylesheet" href="/css/style.css">
</head>

<body>
  <nav>
    <a href="/users">Trở về danh sách users</a>
  </nav>
  <hr>
```

Tạo file mới `views/partials/footer.ejs` , dán nội dung:

```
<hr>
<footer>Nodejs + EJS</footer>
<script src="/js/main.js"></script>
</body>

</html>
```

Tạo file mới `views/users/index.ejs` (trang danh sách), dán nội dung:

```
<%- include('../partials/header.ejs')%>

<div id="test">DANH SÁCH USERS:</div>
<ul>
  <% users.forEach(user=> { %>
    <li>
      <a href="/users/<%= user.id%>">
        <%= user.name%>
      </a>
    </li>
  <%}) %>
</ul>

<%- include('../partials/footer.ejs')%>
```

Tạo file mới `views/users/detail.ejs` (trang chi tiết), dán nội dung:

```
<%- include('../partials/header.ejs')%>

THÔNG TIN USER:
<br>
Name: <%= user.name%><br>
Age: <%= user.age%><br>

<%- include('../partials/footer.ejs')%>
```

Cú pháp EJS:

Cú pháp	Ý nghĩa	Ví dụ
<%= ... %>	In giá trị ra HTML (có escape HTML)	<%= user.name %>
<%- ... %>	In giá trị KHÔNG escape (dùng cho include)	<%- include('header.ejs') %>
<% ... %>	Chạy JavaScript (không in ra)	<% users.forEach(...) %>

Bước 7: Tạo file tĩnh - CSS, JS, ảnh (📁 TẠO FILE)

Tạo file mới `public/css/style.css` , dán nội dung:

```
#test {
  color: red;
}
```

Tạo file mới `public/js/main.js` , dán nội dung:

```
alert('from main.js')
```

Thêm 1 file hình bất kỳ vào `public/images/logo.png` (có thể tải logo nhỏ nào đó).

Bước 8: Tạo file chính (📄 TẠO FILE)

Tạo file mới `index.js` ở thư mục gốc project, dán nội dung:

```
import express from 'express'
import { fileURLToPath } from 'url'
import path from 'path'
import userRouter from './routes/user.route.js'

// Tái tạo __filename và __dirname cho ES Module (xem giải thích Bài 2.2-path).
const __filename = fileURLToPath(import.meta.url)
const __dirname = path.dirname(__filename)

const app = express()
app.use(express.json())

// express.static(folder) = middleware phục vụ FILE TĨNH (HTML, CSS, JS, ảnh...).
// File 'public/css/style.css' → URL '/css/style.css' (bỏ tiền tố 'public').
app.use(express.static(path.join(__dirname, 'public')))

app.use('/users', userRouter)

// CẤU HÌNH VIEW ENGINE
// 'view engine' = 'ejs' → khi gọi res.render('users/index'), Express tự tìm
// 'views/users/index.ejs'.
app.set('view engine', 'ejs')

// Chỉ định thư mục chứa file template. Mặc định là 'views' nhưng nên set tường
// minh.
app.set('views', path.join(__dirname, 'views'))

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Giải thích:

- `app.set('view engine', 'ejs')` - Đăng ký EJS làm template engine. Khi gọi `res.render('users/index')`, Express sẽ tự tìm file `views/users/index.ejs`.
- `app.set('views', path.join(__dirname, 'views'))` - Chỉ định thư mục chứa template.
- `app.use(express.static(path.join(__dirname, 'public')))` - Phục vụ file tĩnh. File `public/css/style.css` sẽ truy cập được qua URL `/css/style.css`.
- Cần tạo lại `__dirname` bằng `fileURLToPath` + `path.dirname` vì ES Module không có sẵn biến này (đã học ở Chương 2).

Bước 9: Chạy server (💻 TERMINAL)

```
npm run dev
```

Bước 10: Test (🌐 TRÌNH DUYỆT)

Mở trình duyệt, truy cập: `http://localhost:3000/users`

Kết quả:

- Hiển thị trang HTML với tiêu đề "DANH SÁCH USERS:" (màu đỏ do CSS)
- Danh sách 2 user dạng link: tuan, vu
- Navbar với link "Trở về danh sách users"
- Alert "from main.js" hiện lên (do file JS client)

Click vào tên user (vd: "tuan"), trình duyệt chuyển đến: `http://localhost:3000/users/1`

Kết quả:

- Hiển thị: "THÔNG TIN USER:" - Name: tuan - Age: 20

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: Cannot find module 'ejs'	Chưa cài EJS	Chạy <code>npm install ejs</code>
Error: Failed to lookup view "users/index"	Sai đường dẫn views hoặc thiếu <code>app.set('views', ...)</code>	Kiểm tra thư mục <code>views/users/index.ejs</code> tồn tại. Kiểm tra <code>app.set('views', ...)</code> trong <code>index.js</code>
CSS/JS/hình không load được	Thiếu <code>express.static</code> hoặc sai đường dẫn	Đảm bảo có <code>app.use(express.static(...))</code> và file nằm trong thư mục <code>public/</code>
ReferenceError: users is not defined trong EJS	Không truyền dữ liệu vào <code>res.render()</code>	Đảm bảo truyền data: <code>res.render('users/index', { users })</code>
Partial không hiển thị	Sai đường dẫn include	Kiểm tra đường dẫn tương đối: <code><%- include('../partials/header.ejs') %></code>

Chương 6: Middleware

Bài 6.2-code-log: Middleware Ghi Log Request

Mục tiêu

- Hiểu middleware là gì và cách hoạt động
- Tạo middleware toàn cục (global middleware)
- Hiểu vai trò của hàm `next()`

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 6.2.code-log
cd 6.2.code-log
npm init -y
npm install express
npm install nodemon --save-dev
```

Bước 1b: Cấu hình `package.json` (📝 SỬA FILE)

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Viết code (📝 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
import express from 'express'

const app = express()

// MIDDLEWARE GLOBAL: chạy cho MỌI request đến server.
// 3 tham số bắt buộc theo thứ tự:
//   req - thông tin request
//   res - response object
//   next - hàm chuyển quyền sang middleware/route tiếp theo
app.use((req, res, next) => {
  // req.method = "GET", "POST", "PUT", "DELETE"...
  // req.url = đường dẫn được request (vd: "/users?id=1")
  console.log(`Method: ${req.method} - URL: ${req.url}`);

  // BẮT BUỘC gọi next() để request đi tiếp đến route handler.
  // Nếu QUÊN next() → request bị TREO, client chờ mãi không có response.
  next();
})

app.get('/', (req, res) => {
  res.send('Hello world!');
})

app.get('/users', (req, res) => {
```

```
    res.send('Lấy thông tin khách hàng.')
  })

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Giải thích:

- **Middleware** là hàm chạy TRƯỚC khi request đến route handler. Nó nhận 3 tham số: `req` , `res` , `next` .
- **`app.use((req, res, next) => {...})`** - Đăng ký middleware toàn cục, chạy cho MỌI request đến server.
- **`next()`** - Gọi hàm này để chuyển quyền xử lý sang middleware/route tiếp theo. Nếu KHÔNG gọi `next()` , request sẽ bị "treo" - client chờ mãi không nhận được response.

Request → Middleware (log) → next() → Route Handler → Response

Bước 3: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 4: Test (🌐 TRÌNH DUYỆT)

Truy cập `http://localhost:3000` rồi `http://localhost:3000/users` trên trình duyệt.

Quan sát Terminal đang chạy server, sẽ hiển thị:

```
Server is running ...
Method: GET - URL: /
Method: GET - URL: /users
```

Mỗi request đều bị middleware "chặn" và ghi log trước khi đến route handler.

Bài 6.2-code-role-admin: Middleware Kiểm Tra Quyền (Authorization)

Mục tiêu

- Tạo middleware kiểm tra quyền truy cập (authorization)
- Áp dụng middleware cho từng route riêng lẻ (route-specific middleware)
- Trả về HTTP status code phù hợp (401 Unauthorized)

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 6.2.code-role-admin
cd 6.2.code-role-admin
npm init -y
npm install express
npm install nodemon --save-dev
```

Bước 1b: Cấu hình `package.json` (🔧 SỬA FILE)

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Viết code (🔧 TẠO FILE)

Tạo file mới `index.js`, dán nội dung:

```
import express from 'express'

const app = express()

// Middleware tự định nghĩa: kiểm tra quyền admin qua query param.
// Demo đơn giản - trong thực tế nên kiểm tra qua session/JWT (sẽ học Chương 10).
const checkAdmin = (req, res, next) => {
  const role = req.query.role; // Lấy role từ URL: ?role=admin
  if (role === 'admin') {
    next() // Cho qua → vào route handler tiếp theo
  }
  // ⚠ CẢNH BÁO LỖI: code này KHÔNG có 'return' trước res.status(401).
  // → Nếu role === 'admin', vừa next() VỪA gửi 401 → lỗi "Cannot set headers
  // after sent".
  // → Cách đúng là: if (role === 'admin') { return next() }
  res.status(401).json({ error: 'Unauthorized' })
}

// Route '/' KHÔNG có middleware → public, ai cũng vào được.
app.get('/', (req, res) => {
  res.send('Hello world!')
})

// Route '/dashboard' CÓ middleware checkAdmin chạy TRƯỚC handler chính.
// Cú pháp: app.get(path, middleware1, middleware2, ..., finalHandler)
app.get('/dashboard', checkAdmin, (req, res) => {
  res.send('Welcome!')
})

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Giải thích:

- checkAdmin** - Middleware tự viết, kiểm tra query param `role` có phải `admin` không.
- Route-specific middleware** - Thay vì dùng `app.use()` (áp dụng cho tất cả), middleware `checkAdmin` chỉ được gắn vào route `/dashboard` bằng cách truyền làm tham số thứ 2 của `app.get()`.
- res.status(401)** - Trả về mã 401 (Unauthorized) nếu không phải admin.

GET /	→ Không qua middleware → Response trực tiếp
GET /dashboard	→ checkAdmin → (nếu admin) → next() → Response

→ (nếu không) → 401 Unauthorized

Bước 3: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 4: Test (🌐 TRÌNH DUYỆT)

Test 1 - Trang public (không cần quyền):

```
http://localhost:3000
```

Kết quả: Hello world!

Test 2 - Truy cập dashboard KHÔNG có quyền:

```
http://localhost:3000/dashboard
```

Kết quả: {"error":"Unauthorized"} (status 401)

Test 3 - Truy cập dashboard VỚI quyền admin:

```
http://localhost:3000/dashboard?role=admin
```

Kết quả: Welcome!

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Request bị "treo", không có response	Quên gọi <code>next()</code> trong middleware	Luôn gọi <code>next()</code> hoặc gửi response (<code>res.send()</code> , <code>res.json()</code>)
Middleware chạy cho tất cả route	Dùng <code>app.use(middleware)</code> thay vì gắn vào route cụ thể	Dùng <code>app.get('/path', middleware, handler)</code> nếu chỉ muốn áp dụng cho 1 route
Cannot set headers after they are sent	Vừa gọi <code>next()</code> vừa gửi response	Dùng <code>return next()</code> hoặc <code>return res.status(401)...</code> để dừng hàm ngay

Chương 7: MongoDB & Mongoose

Yêu cầu: Cần có tài khoản MongoDB Atlas (miễn phí) tại <https://cloud.mongodb.com>. Tạo cluster và lấy connection string.

Bài 7.3: Giới Thiệu Mongoose - Kết Nối MongoDB

Mục tiêu

- Kết nối Node.js với MongoDB Atlas qua Mongoose
- Định nghĩa Schema và Model
- Thực hiện thao tác Create & Read cơ bản
- Sử dụng biến môi trường với `dotenv`

Bước 1: Tạo project và cài đặt (🖥️ TERMINAL)

```
mkdir 7.3.mongoose
cd 7.3.mongoose
npm init -y
# Cài 3 package production cho làm việc với MongoDB:
#   express - framework web
#   mongoose - thư viện ODM kết nối MongoDB
#   dotenv - đọc biến môi trường từ .env
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Bước 1b: Cấu hình `package.json` (📄 SỬA FILE)

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 2: Cấu hình biến môi trường (📄 TẠO FILE)

Tạo file mới `.env` ở thư mục gốc project, dán nội dung:

```
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/?
appName=Cluster0
```

Thay `username`, `password` và `cluster0.xxxxx` bằng thông tin MongoDB Atlas của bạn.

Tạo file mới `.env.example` (📄 TẠO FILE - file mẫu cho người khác clone project), dán nội dung:

```
MONGODB_URI=mongodb+srv://username:password@cluster0.takqys0.mongodb.net/?
appName=Cluster0
```

Tạo file mới `.gitignore` (📄 TẠO FILE), dán nội dung:

```
node_modules
.env
```

Quan trọng: File `.env` chứa mật khẩu database, KHÔNG ĐƯỢC đưa lên Git.

Bước 3: Tạo kết nối database

Bước 3a (🖥️ TERMINAL): Tạo thư mục `config` :

```
mkdir config
```

Bước 3b (📄 TẠO FILE): Tạo file mới `config/db.js` , dán nội dung:

```
// Mongoose là thư viện ODM (Object Data Modeling) giúp làm việc với MongoDB qua JavaScript.
import mongoose from 'mongoose'

// Tách hàm kết nối ra file riêng để dễ tái sử dụng và quản lý.
const connectDB = async () => {
  try {
    // mongoose.connect(uri) trả về Promise → cần await.
    // process.env.MONGODB_URI lấy giá trị từ file .env (đã dotenv.config()).
    await mongoose.connect(process.env.MONGODB_URI)
    console.log('Kết nối MongoDB thành công.');
```

Giải thích:

- `mongoose.connect()` - Kết nối đến MongoDB bằng connection string từ biến môi trường.
- Dùng `async/await` vì kết nối database là thao tác bất đồng bộ.
- `try/catch` để bắt lỗi nếu kết nối thất bại.

Bước 4: Tạo Model (Schema)

Bước 4a (🖥️ TERMINAL): Tạo thư mục `models` :

```
mkdir models
```

Bước 4b (📄 TẠO FILE): Tạo file mới `models/student.model.js` , dán nội dung:

```
import mongoose from "mongoose";

// new mongoose.Schema({...}) định nghĩa CẤU TRÚC dữ liệu (giống "bảng" trong SQL).
// Schema quy định: field nào có, kiểu dữ liệu, validation, mặc định...
const studentSchema = new mongoose.Schema({
  // required: true → bắt buộc nhập, không có sẽ báo lỗi ValidationError.
  name: { type: String, required: true },
  // unique: true → giá trị KHÔNG được trùng (Mongoose tạo unique index trên field này).
  studentId: { type: String, required: true, unique: true },
```

```

    // default → nếu không truyền, tự động lấy giá trị mặc định.
    major: { type: String, default: 'Công nghệ thông tin' },
    // min/max → giới hạn giá trị (chỉ áp dụng với Number).
    gpa: { type: Number, min: 0, max: 10 }
  })

// mongoose.model(tenModel, schema) tạo MODEL – class để thao tác với database.
// Tên 'Student' (số ít, viết hoa) → Mongoose tự tạo collection 'students' (số nhiều, viết thường).
const Student = mongoose.model('Student', studentSchema)

export default Student

```

Giải thích Schema:

Field	Type	Validation	Ý nghĩa
name	String	required: true	Bắt buộc nhập
studentId	String	required: true, unique: true	Bắt buộc và không trùng
major	String	default: 'Công nghệ thông tin'	Nếu không nhập thì lấy giá trị mặc định
gpa	Number	min: 0, max: 10	Chỉ chấp nhận từ 0 đến 10

- **Schema** định nghĩa cấu trúc dữ liệu (như bảng trong SQL).
- **Model** là class để tương tác với collection trong MongoDB.
- `mongoose.model('Student', studentSchema)` tạo collection tên `students` (tự động chuyển sang lowercase + thêm "s").

Bước 5: Tạo file chính (📝 TẠO FILE)

Tạo file mới `index.js` ở thư mục gốc project, dán nội dung:

```

import express from 'express'
import dotenv from 'dotenv'
import connectDB from './config/db.js'
import mongoose from 'mongoose'
import Student from './models/student.model.js'

// dotenv.config() đọc file .env, nạp các biến vào process.env.
// PHẢI gọi TRƯỚC khi sử dụng process.env.XXX (như trong connectDB).
dotenv.config()

// Gọi hàm kết nối DB (không await ở đây – server vẫn start, kết nối chạy song song).
connectDB()

const app = express()
app.use(express.json())

// Endpoint kiểm tra trạng thái kết nối DB – hữu ích khi debug.
app.get('/status', (req, res) => {

```

```

// mongoose.connection.readyState trả về số 0-3, 99 tương ứng trạng thái.
const state = mongoose.connection.readyState
const map = {
  0: 'Disconnected',
  1: 'Connected',
  2: 'Connecting',
  3: 'Disconnecting',
  99: 'Uninitialized'
}
res.json({ db: map[state] })
})

// READ: Lấy danh sách sinh viên
app.get('/api/students', async (req, res) => {
  try {
    // Student.find() (không tham số) → LẤY TẤT CẢ documents trong collection 'students'.
    // Trả về Promise → cần await.
    const students = await Student.find()
    res.json(students)
  } catch (error) {
    console.log('Lỗi lấy danh sách sinh viên: ', error.message);
    res.json({ error: error.message })
  }
})

// CREATE: Tạo sinh viên mới
app.post('/api/students', async (req, res) => {
  try {
    // Bước 1: Tạo instance mới từ Model với dữ liệu request body.
    // (chưa lưu DB, chỉ là object trong RAM)
    const student = new Student(req.body)
    // Bước 2: .save() lưu vào DB. Tự động chạy validation từ Schema.
    await student.save()
    // 201 Created – thông báo đã tạo mới thành công.
    res.status(201).json(student)
  } catch (error) {
    // Lỗi có thể là: ValidationError (thiếu field bắt buộc), duplicate key (studentId trùng)...
    console.log('Lỗi thêm sinh viên: ', error.message);
    res.json({ error: error.message })
  }
})

app.listen(3000, () => {
  console.log('Server is running ...');
})

```

Giải thích:

- `dotenv.config()` - Đọc file `.env` và nạp các biến vào `process.env`.
- `connectDB()` - Gọi hàm kết nối database đã tạo ở bước 3.

- **Student.find()** - Lấy tất cả documents từ collection `students` .
- **new Student(req.body) + student.save()** - Tạo document mới từ dữ liệu request body và lưu vào database.
- **res.status(201)** - HTTP 201 (Created) cho biết tạo mới thành công.
- **/status** - Endpoint kiểm tra trạng thái kết nối database.

Bước 6: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Khi server khởi động thành công, Terminal hiện:

```
[nodemon] starting `node index.js`  
Server is running ...  
Kết nối MongoDB thành công.
```

Bước 7: Test API (🖥️ TERMINAL MỚI)

Mở Terminal MỚI (giữ Terminal cũ đang chạy server).

Test 1 - Kiểm tra kết nối DB:

```
curl http://localhost:3000/status
```

Kết quả: `{"db":"Connected"}`

Test 2 - Tạo sinh viên:

```
curl -X POST http://localhost:3000/api/students \  
-H "Content-Type: application/json" \  
-d '{"name": "Nguyen Van A", "studentId": "SV001", "gpa": 8.5}'
```

Kết quả:

```
{  
  "_id": "664...",  
  "name": "Nguyen Van A",  
  "studentId": "SV001",  
  "major": "Công nghệ thông tin",  
  "gpa": 8.5,  
  "__v": 0  
}
```

Lưu ý: `major` tự động được gán giá trị mặc định, `_id` do MongoDB tự tạo.

Test 3 - Lấy danh sách sinh viên:

```
curl http://localhost:3000/api/students
```

👁️ Bạn có thể vào MongoDB Atlas → Database → Browse Collections để xem document vừa được tạo.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
MongoServerError: bad auth	Sai username/password MongoDB	Kiểm tra lại thông tin trong .env
MongoNetworkError: connect ECONNREFUSED	Không kết nối được MongoDB Atlas	Kiểm tra kết nối internet. Vào MongoDB Atlas -> Network Access -> thêm IP 0.0.0.0/0 (cho phép tất cả)
MongooseError: Operation ... timed out	Kết nối quá chậm/timeout	Kiểm tra internet, hoặc whitelist IP trong MongoDB Atlas
E11000 duplicate key error	studentId bị trùng (đã tồn tại)	Dùng studentId khác, hoặc xóa document cũ
ValidationError: name: Path 'name' is required	Thiếu field bắt buộc	Đảm bảo gửi đủ các field có required: true
process.env.MONGODB_URI là undefined	Thiếu dotenv.config() hoặc file .env	Kiểm tra đã gọi dotenv.config() và file .env tồn tại

Bài 7.4: Thực Hành - CRUD Product Với Mongoose (MVC)

Mục tiêu

- Xây dựng REST API CRUD hoàn chỉnh kết nối MongoDB thật
- Áp dụng kiến trúc MVC với Mongoose
- Sử dụng các Mongoose methods: `find`, `findById`, `findByIdAndUpdate`, `findByIdAndDelete`
- Option `timestamps` tự động theo dõi thời gian tạo/cập nhật

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 7.4.thuchanh-product
cd 7.4.thuchanh-product
npm init -y
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Bước 1b: Cấu hình `package.json` (📝 SỬA FILE)

Thêm `"type": "module"` và script `"dev": "nodemon index.js"` vào `package.json`.

Bước 1c: Tạo file `.env` và `.gitignore` (📝 TẠO FILE)

Tạo `.env`, `.env.example`, `.gitignore` giống bài 7.3 (xem lại nội dung ở Bài 7.3 - Bước 2).

Bước 1d: Tạo cấu trúc thư mục (🖥️ TERMINAL)

```
mkdir config models controllers routes
```

Cấu trúc project bạn sẽ có:

```
7.4.thuchanh-product/  
├─ index.js  
├─ .env  
├─ config/  
│   └─ db.js  
├─ models/  
│   └─ product.model.js  
├─ controllers/  
│   └─ product.controller.js  
└─ routes/  
    └─ product.route.js
```

Bước 2: Tạo kết nối database (🔗 TẠO FILE)

Tạo file mới `config/db.js` , dán nội dung:

```
import mongoose from "mongoose";  
  
const connectDB = async () => {  
  try {  
    await mongoose.connect(process.env.MONGODB_URI)  
    console.log('Kết nối MongoDB thành công');  
  } catch (error) {  
    console.error('Lỗi kết nối MongoDB: ', error.message)  
    // process.exit() - DỪNG hoàn toàn ứng dụng nếu không kết nối được DB.  
    // Lý do: hầu hết tính năng phụ thuộc DB, chạy tiếp cũng vô nghĩa.  
    process.exit()  
  }  
}  
  
export default connectDB
```

`process.exit()` - Dừng ứng dụng nếu không kết nối được database (vì không thể hoạt động nếu không có DB).

Bước 3: Tạo Model (🔗 TẠO FILE)

Tạo file mới `models/product.model.js` , dán nội dung:

```
import mongoose from 'mongoose'  
  
const productSchema = new mongoose.Schema({  
  name: { type: String, required: true },  
  price: { type: Number, min: 0 },  
  category: { type: String },  
  inStock: { type: Boolean, default: true }  
})
```

```

},
// THAM SỐ THỨ 2 của Schema là OPTIONS.
// { timestamps: true } → Mongoose TỰ ĐỘNG thêm 2 field:
//   createdAt – thời gian tạo document (set 1 lần khi insert)
//   updatedAt – thời gian update gần nhất (tự cập nhật khi save/findByIdAndUpdate)
{ timestamps: true })

const Product = mongoose.model('Product', productSchema)

export default Product

```

`{ timestamps: true }` - Mongoose tự động thêm 2 field:

- `createdAt` - Thời điểm tạo document
- `updatedAt` - Thời điểm cập nhật gần nhất

Bước 4: Tạo Controller (📁 TẠO FILE)

Tạo file mới `controllers/product.controller.js`, dán nội dung:

```

import Product from "../models/product.model.js";

// READ ALL: Lấy tất cả products từ MongoDB.
export const getAll = async (req, res) => {
  try {
    // Product.find() trả về Promise → cần await.
    // Tất cả Mongoose query đều BẮT ĐỒNG BỘ (vì phải đợi DB phản hồi qua mạng).
    const products = await Product.find()
    res.json(products)
  } catch (error) {
    // Status 500 = Internal Server Error (lỗi từ phía server, không phải
    client).
    res.status(500).json({ error: error.message })
  }
}

// READ ONE: Lấy 1 product theo _id (MongoDB ObjectId).
export const getOne = async (req, res) => {
  try {
    // findById(id) là shortcut của findOne({_id: id}).
    // id phải đúng định dạng ObjectId (24 ký tự hex), nếu sai → throw
    CastError.
    const product = await Product.findById(req.params.id)
    if (!product) {
      // Không tìm thấy → trả 404 (khác với lỗi 500 – lỗi hệ thống).
      return res.status(404).json({ error: 'Not found' })
    }
    res.json(product)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

```

```

// CREATE: Tạo product mới.
export const create = async (req, res) => {
  try {
    const product = new Product(req.body) // Tạo instance từ body request
    await product.save() // Lưu vào DB (chạy validation)
    res.status(201).json(product) // 201 Created
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

// UPDATE: Cập nhật product theo id.
export const update = async (req, res) => {
  try {
    // findByIdAndUpdate(id, dataMoi, options) - tìm theo id và update trong 1
    // lệnh.
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      // OPTIONS:
      // new: true → TRẢ VỀ document SAU KHI update (mặc định trả
      // document CŨ).
      // runValidators: true → CHẠY validation từ Schema khi update (mặc
      // định KHÔNG chạy).
      { new: true, runValidators: true }
    )
    if (!product) {
      return res.status(404).json({ error: 'Not found' })
    }
    res.json(product)
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

// DELETE: Xóa product theo id.
export const remove = async (req, res) => {
  try {
    // findByIdAndDelete(id) - xóa document, trả về document đã xóa hoặc null.
    const product = await Product.findByIdAndDelete(req.params.id)
    if (!product) {
      return res.status(404).json({ error: 'Not found' })
    }
    res.json(product) // Trả về product vừa xóa để client biết
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

```

Giải thích các Mongoose methods:

Method	Ý nghĩa	Ghi chú
<code>Product.find()</code>	Lấy tất cả documents	Trả về mảng
<code>Product.findById(id)</code>	Tìm 1 document theo <code>_id</code>	Trả về object hoặc null
<code>new Product(data) + .save()</code>	Tạo và lưu document mới	Chạy validation
<code>Product.findByIdAndUpdate(id, data, options)</code>	Tìm và cập nhật	<code>new: true</code> trả về document sau khi cập nhật
<code>Product.findByIdAndDelete(id)</code>	Tìm và xoá	Trả về document đã xoá hoặc null

- `{ new: true }` - Mặc định `findByIdAndUpdate` trả về document CŨ (trước khi update). Thêm option này để trả về document MỚI.
- `{ runValidators: true }` - Chạy validation schema khi update (mặc định không chạy).

Bước 5: Tạo Route (📝 TẠO FILE)

Tạo file mới `routes/product.route.js`, dán nội dung:

```
import { Router } from "express";
import { getAll, getOne, create, update, remove } from
"../controllers/product.controller.js";

const router = Router()

router.get('/', getAll)
router.get('/:id', getOne)
router.post('/', create)
router.put('/:id', update)
router.delete('/:id', remove)

export default router
```

Bước 6: Tạo file chính (📝 TẠO FILE)

Tạo file mới `index.js` ở thư mục gốc project, dán nội dung:

```
import express from 'express'
import dotenv from 'dotenv'
import mongoose from 'mongoose'
import connectDB from './config/db.js'
import productRouter from './routes/product.route.js'

dotenv.config() // PHẢI gọi TRƯỚC các xử lý dùng process.env

const app = express()
```

```

app.use(express.json())

// await connectDB() – DỪNG khởi động server CHO TỚI KHI DB kết nối xong.
// Top-level await chỉ hoạt động khi file là ES Module ("type": "module").
// Lợi ích: tránh tình trạng client gọi API trước khi DB sẵn sàng.
await connectDB()

app.use('/api/products', productRouter)

// Route kiểm tra trạng thái DB (giống Bài 7.3).
app.get('/status', (req, res) => {
  const state = mongoose.connection.readyState
  const map = {
    0: 'Disconnected',
    1: 'Connected',
    2: 'Connecting',
    3: 'Disconnecting',
    99: 'uninitialized'
  }
  res.json({ db: map[state] })
})

app.listen(3000, () => {
  console.log('Server is running ...');
})

```

`await connectDB()` - Chờ kết nối database thành công **TRƯỚC** khi server bắt đầu nhận request.

Bước 7: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 8: Test API (🖥️ TERMINAL MỚI)

Mở Terminal MỚI để test (giữ Terminal cũ đang chạy server).

```

# Test 1 – Tạo product mới
curl -X POST http://localhost:3000/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Laptop", "price": 1500, "category": "Electronics"}'

# Test 2 – Lấy tất cả products
curl http://localhost:3000/api/products

# Test 3 – Lấy theo id (THAY <id> bằng _id thật từ kết quả Test 1)
curl http://localhost:3000/api/products/<id>

# Test 4 – Cập nhật product
curl -X PUT http://localhost:3000/api/products/<id> \
  -H "Content-Type: application/json" \

```

```
-d '{"price": 1200}'
```

```
# Test 5 - Xóa product
```

```
curl -X DELETE http://localhost:3000/api/products/<id>
```

Sau Test 1, copy giá trị `_id` (vd: `665a...`) từ kết quả trả về để dùng cho Test 3, 4, 5.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
CastError: Cast to ObjectId failed	ID không đúng format MongoDB ObjectId (24 ký tự hex)	Dùng đúng <code>_id</code> từ response khi tạo product
Dữ liệu vẫn còn sau restart	Khác với các bài trước, giờ dữ liệu lưu trong MongoDB thật	Đây là hành vi đúng - dữ liệu persistent
timestamps không xuất hiện	Thiếu { timestamps: true } trong schema	Thêm option { timestamps: true } làm tham số thứ 2 của <code>new mongoose.Schema()</code>

Chương 9: MongoDB Nâng Cao

Bài 9.4: Query Operators - Toán Tử Truy Vấn MongoDB

Mục tiêu

- Sử dụng các toán tử truy vấn MongoDB: `$gt`, `$in`, `$and`, `$regex`
- Schema với nested object và mảng
- Seed dữ liệu mẫu vào database

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 9.4.mongo-operators
cd 9.4.mongo-operators
npm init -y
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Bước 1b: Cấu hình project (📄 SỬA/TẠO FILE)

Cấu hình `"type": "module"`, script `"dev": "nodemon index.js"` trong `package.json`. Tạo `.env`, `.gitignore` như các bài Chương 7.

Bước 1c: Tạo cấu trúc thư mục (🖥️ TERMINAL)

```
mkdir config models controllers routes
```

Bước 2: Tạo kết nối database (📄 TẠO FILE)

Tạo file mới `config/db.js` (nội dung giống bài 7.3).

Bước 3: Tạo Model với nested object & mảng (📄 TẠO FILE)

Tạo file mới `models/product.model.js`, dán nội dung:

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    // index: true → tự tạo INDEX trên field 'name' để tìm kiếm theo name NHANH
    // hơn.
    // (Tương tự index trong SQL – như mục lục sách giúp tra cứu nhanh.)
    name: { type: String, required: true, index: true },
    price: { type: Number, min: 0 },
    category: { type: String },
    inStock: { type: Boolean, default: true },
    // NESTED OBJECT: address là object con chứa city và country.
    // Truy cập: product.address.city
    address: {
```

```

        city: { type: String },
        country: { type: String }
    },
    // ARRAY OF STRINGS: tags là mảng các chuỗi.
    // Vd: tags: ["sale", "new", "hot"]
    tags: [{ type: String }]
}

)

// COMPOUND INDEX (index kết hợp 2 field):
// inStock: 1 → tăng dần (true sau false)
// name: -1 → giảm dần (Z → A)
// Hữu ích khi query có ĐỒNG THỜI 2 điều kiện (vd: find({inStock: true}).sort({name: -1}))
productSchema.index({ inStock: 1, name: -1 })

const Product = mongoose.model('Product', productSchema)

export default Product

```

Giải thích:

- **address** - Nested object (object lồng nhau) chứa **city** và **country** .
- **tags: [{ type: String }]** - Mảng các string.
- **index: true** - Tạo index trên field **name** để tìm kiếm nhanh hơn.
- **productSchema.index({ inStock: 1, name: -1 })** - Tạo compound index (index kết hợp): **inStock** tăng dần, **name** giảm dần.

Bước 4: Tạo Controller với Seed & Query (👉 TẠO FILE)

Tạo file mới `controllers/product.controllers.js` , dán nội dung:

```

import Product from "../models/product.model.js";

// SEED: tạo dữ liệu mẫu để test (thường dùng 1 lần khi setup project).
export const seed = async (req, res) => {
    try {
        // deleteMany() (không tham số) → XÓA TẤT CẢ documents.
        // CẢN THẬN: thao tác này không thể undo. Chỉ dùng cho dev/test.
        await Product.deleteMany()
        // insertMany([...]) → thêm NHIỀU documents trong 1 lệnh (nhanh hơn save() từng cái).
        await Product.insertMany([
            { name: "iPhone 15", price: 999, category: "Electronics", inStock: true,
            address: { city: "hanoi", country: "vietnam" }, tags: ["sale", "new"] },
            { name: "iPhone 14", price: 799, category: "Electronics", inStock:
            false, address: { city: "hanoi", country: "vietnam" }, tags: ["sale"] },
            { name: "Laptop pro", price: 1500, category: "Electronics", inStock:
            true, address: { city: "hcm", country: "vietnam" }, tags: ["sale", "new"] },
            { name: "Keyboard", price: 100, category: "Accessories", inStock: true,
            tags: ["sale"] },
            { name: "Mouse", price: 50, category: "Accessories", inStock: false },

```

```

    })
    res.json({ message: 'Seed data xong' })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

// TEST: ví dụ về các QUERY OPERATORS – bỏ comment từng dòng để thử.
export const test = async (req, res) => {
  try {
    const products = await Product.find(
      // $gt = "greater than" (lớn hơn): giá > 799
      // { price: { $gt: 799 } }

      // $in = "in array" (nằm trong danh sách)
      // { category: { $in: ['Electronics', 'Accessories'] } }

      // $and = TẤT CẢ điều kiện phải đúng
      // {
      //   $and: [
      //     { price: { $gt: 100 } },
      //     { inStock: true }
      //   ]
      // }

      // $regex = tìm theo PATTERN (như Find trong Word).
      // $options: 'i' = case-INSENSITIVE (không phân biệt hoa thường)
      // → vẫn match "iPhone", "iphone", "IPHONE"...
      { name: { $regex: "iPhone", $options: 'i' } }
    )
    res.json(products)
  } catch (error) {
    res.status(500).json({ message: error.message })
  }
}

```

Giải thích các Query Operators:

Operator	Ý nghĩa	Ví dụ	Kết quả
\$gt	Greater than (lớn hơn)	{ price: { \$gt: 799 } }	Sản phẩm giá > 799
\$in	Nằm trong danh sách	{ category: { \$in: ['Electronics'] } }	Sản phẩm thuộc Electronics
\$and	Và (tất cả điều kiện đúng)	{ \$and: [{price: {\$gt:100}}, {inStock: true}] }	Giá > 100 VÀ còn hàng
\$regex	Tìm theo pattern	{ name: { \$regex: "iPhone", \$options: 'i' } }	Tên chứa "iphone" (không phân biệt hoa/thường)

- `$options: 'i'` - Case-insensitive (không phân biệt chữ hoa/thường).
- `Product.deleteMany()` - Xóa tất cả documents trong collection.
- `Product.insertMany([...])` - Chèn nhiều documents cùng lúc.

Bước 5: Tạo Route và file chính (📝 TẠO FILE)

Tạo file mới `routes/product.route.js`, dán nội dung:

```
import { Router } from 'express';
import { seed, test } from '../controllers/product.controllers.js';

const router = Router()

router.post('/seed', seed)
router.get('/test', test)

export default router
```

Tạo file mới `index.js` ở thư mục gốc project, dán nội dung:

```
import express from 'express'
import dotenv from 'dotenv'
import productRouter from './routes/product.route.js'
import connectDB from './config/db.js'

dotenv.config()
await connectDB()

const app = express()
app.use(express.json())

app.use('/api/products', productRouter)

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 6: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 7: Test (🖥️ TERMINAL MỚI)

Mở Terminal MỚI để test.

```
# Bước 7.1 – Seed dữ liệu mẫu (chỉ chạy 1 lần – sẽ XÓA toàn bộ data cũ rồi thêm 5 product mới)
curl -X POST http://localhost:3000/api/products/seed
```

```
# Bước 7.2 – Test query operators (xem kết quả filter theo regex hiện tại)
curl http://localhost:3000/api/products/test
```

Thử nghiệm thêm: Mở `controllers/product.controllers.js`, bỏ comment từng dòng query khác trong hàm `test()` (như `{ price: { $gt: 799 } }`), lưu file (nodemon sẽ tự restart), rồi gọi lại `curl http://localhost:3000/api/products/test` để xem kết quả khác nhau.

Bài 9.th: Thực Hành - Explain Query, Index & Phân Trang

Mục tiêu

- Seed 100,000 documents để test hiệu năng
- So sánh tốc độ query CÓ index vs KHÔNG có index bằng `explain()`
- Thực hiện phân trang (pagination) với `skip`, `limit`, `sort`

Bước 1: Tạo project (🖥️ TERMINAL)

```
mkdir 9.thuc-hanh
cd 9.thuc-hanh
npm init -y
npm install express mongoose dotenv
npm install nodemon --save-dev
```

Bước 1b: Cấu hình project (📝 SỬA/TẠO FILE)

Cấu hình `"type": "module"`, script `"dev"`, `.env`, `.gitignore` tương tự các bài Chương 7.

Bước 1c: Tạo cấu trúc thư mục (🖥️ TERMINAL)

```
mkdir config models controllers routes
```

Bước 1d: Tạo file `config/db.js` (📝 TẠO FILE)

Tạo file `config/db.js` (nội dung giống bài 7.3).

Bước 2: Tạo Model với duplicate fields - cho so sánh (📝 TẠO FILE)

Tạo file mới `models/product.model.js`, dán nội dung:

```
import mongoose from 'mongoose';

const productSchema = new mongoose.Schema({
  // CẤP FIELD CÓ INDEX (sẽ được index ở dòng productSchema.index bên dưới).
  name: { type: String, required: true },
  category: { type: String },
  price: { type: String, min: 0 },
  inStock: { type: Boolean },

  // CẤP FIELD KHÔNG CÓ INDEX (cố tình tạo để so sánh tốc độ query).
  name2: { type: String, required: true },
```

```

    category2: { type: String },
  }, { timestamps: true })

// Compound index trên name + category để query tổ hợp 2 field này nhanh hơn.
productSchema.index({ name: 1, category: -1 })

const Product = mongoose.model('Product', productSchema)

export default Product

```

`name2` , `category2` là bản sao KHÔNG có index - dùng để so sánh tốc độ với `name` , `category` CÓ index.

Bước 3: Tạo Controller (👉 TẠO FILE)

Tạo file mới `controllers/product.controller.js` , dán nội dung:

```

import Product from '../models/product.model.js';

// SEED: Tạo 100,000 documents để test hiệu năng query.
export const seed = async (req, res) => {
  try {
    await Product.deleteMany() // Xoá toàn bộ data cũ
    await Product.collection.dropIndexes() // Xoá tất cả index cũ → bắt
    đầu sạch

    // Tạo mảng 100,000 phần tử để insertMany 1 lần (nhanh hơn loop save từng
    cái).
    const bulk = []
    for (let i = 0; i < 100000; i++) {
      // i % 2 → kết quả 0 hoặc 1, dùng để chia category xen kẽ.
      const category = i % 2 === 0 ? 'Electronics' : 'Accessories';
      const name = `Product ${i}`
      bulk.push({
        name,
        category,
        name2: name, // Sao chép vào field không-index để so sánh
        category2: category,
        // Math.random() trả về 0-1, * 100000 → 0-100000, Math.floor() → bỏ
        phần lẻ.
        price: Math.floor(Math.random() * 100000),
        inStock: i % 3 === 0 // Cứ 3 sản phẩm có 1 còn hàng
      })
    }
    await Product.insertMany(bulk) // Insert 100K documents 1 lần

    res.json({ message: 'Đã thực hiện seed 100000 documents xong' })
  } catch (error) {
    // (Cố ý để trống catch để demo – thực tế nên xử lý lỗi)
  }
}

```

```

// EXPLAIN: So sánh tốc độ query CÓ index vs KHÔNG có index.
export const explainQuery = async (req, res) => {
  try {
    // Xóa index cũ, tạo lại index trên name+category để chắc chắn có index khi
    test.

    await Product.collection.dropIndexes()
    await Product.collection.createIndex({ name: 1, category: -1 })

    // Query trên field2 (KHÔNG có index) → MongoDB phải COLLSCAN (quét toàn bộ
    collection).
    // .explain('executionStats') yêu cầu trả về THỐNG KÊ thực thi thay vì dữ
    liệu.
    // /^Product/ là regex - tìm chuỗi BẮT ĐẦU bằng "Product" (^ = đầu chuỗi).
    const before = await Product
      .find({ category2: 'Electronics', name2: /^Product/ })
      .explain('executionStats')

    // Query trên field có index → MongoDB dùng IXSCAN (quét qua index, nhanh
    hơn).
    const after = await Product
      .find({ category: 'Electronics', name: /^Product/ })
      .explain('executionStats')

    res.json({
      without_index: {
        stage: before.executionStats.executionStages.stage, //
        "COLLSCAN" hay "IXSCAN"
        docsExamined: before.executionStats.totalDocsExamined, // Số
        docs đã quét
        docsReturned: before.executionStats.totalDocsReturned, // Số
        docs trả về
        executionTimeMillis: before.executionStats.executionTimeMillis, //
        Tổng thời gian (ms)
      },
      with_index: {
        stage: after.executionStats.executionStages.stage,
        docsExamined: after.executionStats.totalDocsExamined,
        docsReturned: after.executionStats.totalDocsReturned,
        executionTimeMillis: after.executionStats.executionTimeMillis,
      }
    })
  } catch (error) {
    res.status(500).json({ error: error.message })
  }
}

// PAGINATE: Phân trang kết quả - kỹ thuật bắt buộc khi data lớn.
export const paginate = async (req, res) => {
  try {
    // parseInt() chuyển string → integer. || 1 nghĩa là nếu undefined/NaN thì
    dùng 1.
    // Math.max(1, x) đảm bảo page tối thiểu là 1 (không cho người dùng nhập 0

```

hoặc âm).

```
const page = Math.max(1, parseInt(req.query.page) || 1)
const limit = Math.max(1, parseInt(req.query.limit) || 10)
// Công thức: trang 1 bỏ qua 0, trang 2 bỏ qua 10, trang 3 bỏ qua 20...
const skip = (page - 1) * limit

// Field để sắp xếp - mặc định 'createdAt' (mới nhất trước).
const sortField = req.query.sort || 'createdAt'
// 1 = tăng dần (ASC), -1 = giảm dần (DESC).
const sortOrder = req.query.order === 'asc' ? 1 : -1
// [sortField] là COMPUTED PROPERTY NAME - tên field lấy động từ biến.
// Vd: nếu sortField='price', sortOrder=1 → sort = {price: 1}
const sort = { [sortField]: sortOrder }

// Filter tìm theo name (nếu có query ?name=xxx), không có thì lấy tất cả.
const filter = req.query.name ? { name: { $regex: req.query.name, $options:
'i' } } : {}

// Promise.all chạy SONG SONG 2 query để nhanh hơn:
// 1. Lấy data theo phân trang
// 2. Đếm tổng số documents (để tính tổng số trang)
// .sort().skip().limit() là CHAIN - thứ tự không quan trọng vì MongoDB
optimize.
const [products, total] = await Promise.all([
  Product.find(filter).sort(sort).skip(skip).limit(limit),
  Product.countDocuments(filter)
])

res.json({
  data: products,
  pagination: {
    total, // Tổng documents
    page, // Trang hiện tại
    limit, // Items mỗi trang
    totalPages: Math.ceil(total / limit), // Tổng số trang (làm
tròn lên)
    hasNext: page < Math.ceil(total / limit), // Còn trang sau không?
    hasPrev: page > 1, // Còn trang trước
không?
  }
})
} catch (error) {
  res.status(500).json({ error: error.message })
}
}
```

Giải thích hàm `explainQuery` :

- `.explain('executionStats')` - Yêu cầu MongoDB trả về thống kê thực thi thay vì dữ liệu.
- **COLLSCAN** - Collection Scan, quét TOÀN BỘ documents (chậm).
- **IXSCAN** - Index Scan, quét qua index (nhanh).

- So sánh `docsExamined` giữa 2 cách: không index phải quét ~100,000 docs, có index chỉ quét vài nghìn.

Giải thích hàm `paginate` :

Tham số	Ý nghĩa	Mặc định
<code>page</code>	Trang hiện tại	1
<code>limit</code>	Số items mỗi trang	10
<code>sort</code>	Field sắp xếp	<code>createdAt</code>
<code>order</code>	Thứ tự (asc/desc)	<code>desc</code>
<code>name</code>	Lọc theo tên (regex)	Không lọc

- `.skip(skip)` - Bỏ qua `skip` documents đầu tiên.
- `.limit(limit)` - Lấy tối đa `limit` documents.
- `.sort(sort)` - Sắp xếp kết quả.
- `Promise.all([...])` - Chạy song song query lấy data và đếm tổng số documents.

Bước 4: Tạo Route và file chính (📁 TẠO FILE)

Tạo file mới `routes/product.route.js`, dán nội dung:

```
import { Router } from "express";
import { explainQuery, paginate, seed } from "../controllers/product.controller.js";

const router = new Router();

router.post('/seed', seed)
router.get('/explain', explainQuery)
router.get('/paginate', paginate)

export default router
```

Tạo file mới `index.js` ở thư mục gốc project, dán nội dung:

```
import express from 'express';
import connectDB from './config/db.js';
import dotenv from 'dotenv';
import productRouter from './routes/product.route.js';

dotenv.config()
await connectDB()
const app = express()
app.use(express.json())

app.use('/api/products', productRouter)

app.listen(3000, () => {
```

```
    console.log('Server is running ...');
  })
```

Bước 5: Chạy server (🖥️ TERMINAL)

```
npm run dev
```

Bước 6: Test (🖥️ TERMINAL MỚI)

Mở Terminal MỚI để test (giữ Terminal cũ đang chạy server).

```
# Test 1 - Seed 100,000 documents (có thể mất 10-30 giây, KIÊN NHẪN CHỜ)
curl -X POST http://localhost:3000/api/products/seed

# Test 2 - So sánh hiệu năng query có index vs không index
curl http://localhost:3000/api/products/explain

# Test 3 - Phân trang: trang 1, mỗi trang 5 item
curl "http://localhost:3000/api/products/paginate?page=1&limit=5"

# Test 4 - Phân trang + lọc theo tên + sắp xếp theo giá tăng dần
# (Dấu ngoặc kép quanh URL là BẮT BUỘC vì URL chứa &)
curl "http://localhost:3000/api/products/paginate?page=1&limit=5&name=Product%201&sort=price&order=asc"
```

%20 là gì? Đây là cách mã hóa dấu CÁCH trong URL (URL encoding). `Product%201` nghĩa là "Product 1".

Kết quả explain (mẫu):

```
{
  "without_index": {
    "stage": "COLLSCAN",
    "docsExamined": 100000,
    "docsReturned": 50000
  },
  "with_index": {
    "stage": "IXSCAN",
    "docsExamined": 50000,
    "docsReturned": 50000
  }
}
```

Không có index: quét 100,000 docs. Có index: quét ít hơn đáng kể.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
-----	-------------	----------

Seed quá lâu hoặc timeout	100,000 documents cần thời gian	Chờ đợi, hoặc giảm số lượng xuống 10,000 để test
explain trả về lỗi	Structure của explain result thay đổi theo phiên bản MongoDB	Kiểm tra phiên bản MongoDB Atlas đang dùng

Chương 10: Xác Thực & Phiên Làm Việc

Bài 10.2: Mã Hóa Mật Khẩu Với Bcrypt

Mục tiêu

- Hiểu tại sao KHÔNG ĐƯỢC lưu mật khẩu dạng plain text
- Sử dụng `bcrypt` để hash mật khẩu khi đăng ký
- Sử dụng `bcrypt.compare()` để xác thực khi đăng nhập

Quy ước ký hiệu trong bài

Để bạn dễ phân biệt việc nào làm ở Terminal, việc nào sửa trong code:

-  **TERMINAL** - Lệnh chạy trong Terminal/PowerShell (đoạn ````bash` hoặc ````text`)
-  **TẠO FILE / SỬA FILE** - Tạo hoặc sửa nội dung file trong VS Code (đoạn ````javascript` , ````json` , ````text`)

Cấu trúc thư mục mà bạn sẽ tạo ra sau bài này

```
10.2-bcrypt/
├─ package.json      ← npm init tự tạo, ta sửa thêm "type": "module" + script "dev"
├─ .env              ← Ta tự tạo, chứa biến môi trường (MONGODB_URI, PORT)
├─ index.js          ← File chính khởi động server
├─ config/
│   └─ db.js         ← Hàm kết nối MongoDB
├─ models/
│   └─ user.model.js ← Schema User
├─ controllers/
│   └─ auth.controller.js ← Hàm xử lý đăng ký, đăng nhập
└─ routes/
    └─ auth.route.js  ← Khai báo route /register, /login
```

Bước 1: Tạo project (TERMINAL)

Mở Terminal, di chuyển tới thư mục bạn muốn lưu project (vd: Desktop), sau đó chạy:

```
# Tạo thư mục project và vào trong
mkdir 10.2-bcrypt
cd 10.2-bcrypt

# Khởi tạo file package.json
npm init -y

# Cài 4 package PRODUCTION (cần khi chạy thật):
#   express  - framework web
#   mongoose - kết nối MongoDB
#   bcrypt   - hash mật khẩu
#   dotenv   - đọc biến môi trường từ file .env
npm install express mongoose bcrypt dotenv
```

```
# Cài 1 package DEV (chỉ cần khi phát triển):  
#   nodemon - tự restart server khi sửa code  
npm install nodemon --save-dev  
  
# Tạo các thư mục con để chứa code theo MVC  
mkdir config models controllers routes
```

Sau khi chạy xong, bạn sẽ thấy: `package.json` , `package-lock.json` , thư mục `node_modules/` và 4 thư mục con vừa tạo.

Bước 1b: Cấu hình `package.json` (✎ SỬA FILE)

Mở file `package.json` trong VS Code, **THÊM** 2 dòng `"type": "module"` và script `"dev"` . Sau khi sửa, file trông như sau:

```
{  
  "name": "10.2-bcrypt",  
  "version": "1.0.0",  
  "main": "index.js",  
  "type": "module",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1",  
    "dev": "nodemon index.js"  
  },  
  "dependencies": {  
    "bcrypt": "...",  
    "dotenv": "...",  
    "express": "...",  
    "mongoose": "..."  
  },  
  "devDependencies": {  
    "nodemon": "..."  
  }  
}
```

Phần số phiên bản `...` để mặc định, không cần sửa.

Bước 1c: Tạo file `.env` (✎ TẠO FILE)

Tạo file mới tên `.env` ở thư mục gốc project (cùng cấp `package.json`). Dán nội dung:

```
PORT=3000  
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/?  
appName=Cluster0
```

Thay `username` , `password` , `cluster0.xxxxx` bằng thông tin MongoDB Atlas của bạn (xem [Chương 0 mục 0.4](#)).

Tạo thêm file `.gitignore` (✎ TẠO FILE):

```
node_modules
.env
```

Đảm bảo `.env` được đưa vào `.gitignore` để không vô tình đẩy mật khẩu DB lên Git.

Bước 2: Tạo kết nối database (📝 TẠO FILE)

Tạo file mới `config/db.js` , dán nội dung:

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log("MongoDB connected");
  } catch (error) {
    console.error("Error connecting to MongoDB:", error);
    process.exit(1);
  }
};

export default connectDB;
```

Bước 3: Tạo User Model (📝 TẠO FILE)

Tạo file mới `models/user.model.js` , dán nội dung:

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true // Không cho phép trùng username
  },
  // QUAN TRỌNG: tên field 'hashedPassword' (không phải 'password')
  // → nhắc nhở rằng KHÔNG BAO GIỜ lưu mật khẩu gốc, chỉ lưu chuỗi đã hash.
  hashedPassword: {
    type: String,
    required: true
  }
}, { timestamps: true }); // Tự thêm createdAt, updatedAt

const User = mongoose.model("User", userSchema);

export default User;
```

Lưu ý: Field tên là `hashedPassword` - nhắc nhở rằng KHÔNG BAO GIỜ lưu mật khẩu gốc.

Bước 4: Tạo Auth Controller (📝 TẠO FILE)

Tạo file mới `controllers/auth.controller.js` , dán nội dung:

```
import bcrypt from "bcrypt";
import User from "../models/user.model.js";

// ĐĂNG KÝ TÀI KHOẢN
export const registerUser = async (req, res) => {
  try {
    // Destructuring lấy 2 field từ body request.
    const { username, password } = req.body;

    // bcrypt.hash(plaintext, saltRounds) – HASH mật khẩu trước khi lưu.
    // saltRounds = 10 → 2^10 = 1024 lần lặp. Số càng cao càng an toàn nhưng
    càng chậm.
    // Kết quả: chuỗi dài ~60 ký tự, vd: "$2b$10$K8f3g..."
    // → MỖI LẦN hash CÙNG 1 mật khẩu cũng cho ra kết quả KHÁC NHAU (nhờ salt
    ngẫu nhiên).
    const hashedPassword = await bcrypt.hash(password, 10);

    // Lưu user mới vào DB với mật khẩu ĐÃ HASH (không lưu plaintext).
    const user = new User({ username, hashedPassword });
    await user.save();
    res.status(201).json({ message: "User registered successfully" });
  } catch (error) {
    res.status(500).json({ message: "Error registering user" });
  }
};

// ĐĂNG NHẬP
export const loginUser = async (req, res) => {
  try {
    const { username, password } = req.body;

    // findOne(filter) trả về DOCUMENT đầu tiên khớp filter, hoặc null nếu không
    có.
    const user = await User.findOne({ username });
    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    // bcrypt.compare(plaintext, hash) – so sánh mật khẩu nhập vào với hash đã
    lưu.
    // Trả về true nếu khớp, false nếu sai.
    // KHÔNG cần tách salt – bcrypt tự lấy salt từ chuỗi hash.
    const isValid = await bcrypt.compare(password, user.hashedPassword);
    if (!isValid) {
      // 401 Unauthorized – chưa xác thực được danh tính.
      return res.status(401).json({ message: "Invalid password" });
    }

    // Đăng nhập thành công – tuy nhiên chưa có cơ chế giữ session.
```

```
// → Lần sau client gọi API vẫn phải đăng nhập lại. Bài 10.3 giải quyết bằng Session.
res.status(200).json({ message: "User logged in successfully" });
} catch (error) {
  res.status(500).json({ message: "Error logging in user" });
}
};
```

Giải thích Bcrypt:

- **bcrypt.hash(password, 10)** - Hash mật khẩu với 10 **salt rounds**. Salt là chuỗi ngẫu nhiên thêm vào trước khi hash, số 10 là độ phức tạp (càng cao càng an toàn nhưng càng chậm).
 - Input: "myPassword123" -> Output: "\$2b\$10\$xK8f3g..." (chuỗi hash dài ~60 ký tự)
 - Cùng một mật khẩu, mỗi lần hash cho ra kết quả KHÁC NHAU (nhờ salt ngẫu nhiên).
- **bcrypt.compare(password, hashedPassword)** - So sánh mật khẩu gốc với hash đã lưu. Trả về **true** nếu khớp, **false** nếu không. KHÔNG CẦN biết salt vì salt được lưu trong chuỗi hash.

Bước 5: Tạo Route (📁 TẠO FILE)

Tạo file mới `routes/auth.route.js`, dán nội dung:

```
import { Router } from "express";
import { registerUser, loginUser } from "../controllers/auth.controller.js";

const router = Router();

router.post("/register", registerUser);
router.post("/login", loginUser);

export default router;
```

Bước 6: Tạo file chính (📁 TẠO FILE)

Tạo file mới `index.js` ở thư mục gốc project (cùng cấp `package.json`), dán nội dung:

```
import express from "express";
import dotenv from "dotenv";
import connectDB from "../config/db.js";
import authRoute from "../routes/auth.route.js";

// Đọc file .env, nạp các biến vào process.env (PHẢI gọi trước khi dùng process.env.XXX).
dotenv.config();
// Đợi kết nối MongoDB xong rồi mới chạy tiếp.
await connectDB();

const app = express();
// Đọc PORT từ .env. Nếu .env không có thì mặc định 3000.
const PORT = process.env.PORT || 3000;

app.use(express.json()); // Bật phân tích JSON body
```

```
app.use("/api/auth", authRoute);           // Gắn router vào prefix /api/auth

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

Bước 7: Chạy server (🖥️ TERMINAL)

Đảm bảo bạn đang ở thư mục `10.2-bcrypt/` , chạy lệnh:

```
npm run dev
```

Khi server khởi động thành công, Terminal hiện:

```
[nodemon] starting `node index.js`
MongoDB connected
Server is running on port 3000
```

Lưu ý: Cứ MỞ Terminal này để server chạy. Mở Terminal MỚI để test (Bước 8). Trong VS Code có thể bấm dấu **+** ở góc phải Terminal để mở Terminal mới.

Bước 8: Test API (🖥️ TERMINAL MỚI)

Mở **Terminal mới** (giữ Terminal cũ đang chạy server) hoặc dùng **Postman / Thunder Client**.

Test 1 - Đăng ký:

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "123456"}'
```

Kết quả mong đợi: `{"message": "User registered successfully"}`

👁️ Sau lệnh này, vào MongoDB Atlas → Database → Browse Collections → bạn sẽ thấy collection `users` có 1 document với `hashedPassword` là chuỗi dạng `$2b$10$xK8f3g...` (KHÔNG PHẢI `123456`).

Test 2 - Đăng nhập đúng mật khẩu:

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "123456"}'
```

Kết quả mong đợi: `{"message": "User logged in successfully"}`

Test 3 - Đăng nhập sai mật khẩu:

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "wrong"}'
```

Kết quả mong đợi: `{"message": "Invalid password"}` (status 401)

Dừng server: Quay lại Terminal đang chạy server, nhấn `Ctrl + C`.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: Cannot find module 'bcrypt'	Chưa cài bcrypt	<code>npm install bcrypt</code>
Lỗi cài bcrypt trên Windows	bcrypt cần build native module	Thử <code>npm install bcryptjs</code> (pure JS, không cần build) rồi đổi <code>import bcrypt from "bcryptjs"</code>
E11000 duplicate key error	Username đã tồn tại	Dùng username khác

Bài 10.3: Session & Middleware Xác Thực

Mục tiêu

- Quản lý phiên đăng nhập (session) với `express-session`
- Lưu session vào MongoDB với `connect-mongo`
- Tạo middleware `requireLogin` bảo vệ route
- Luồng hoàn chỉnh: Đăng ký -> Đăng nhập -> Truy cập trang bảo vệ -> Đăng xuất

Quy ước ký hiệu trong bài

-  **TERMINAL** - *Lệnh chạy trong Terminal/PowerShell*
-  **TẠO FILE / SỬA FILE** - *Tạo hoặc sửa file trong VS Code*

Cấu trúc thư mục mà bạn sẽ tạo ra sau bài này

```
10.3.session-middleware/  
├─ package.json      ← npm init tự tạo, sửa thêm "type": "module" + script "dev"  
├─ .env              ← Tự tạo, chứa MONGODB_URI + SESSION_SECRET  
├─ src/  
│   ├── index.js      ← File chính khởi động server  
│   └─ config/  
│       ├── db.js      ← Kết nối MongoDB  
│       └─ session.js  ← Cấu hình express-session + MongoStore  
│   └─ models/  
│       └─ user.model.js ← Schema User  
│   └─ controllers/  
│       └─ user.controller.js ← Đăng ký, đăng nhập, profile, logout  
│   └─ middlewares/  
│       └─ requireLogin.js ← Middleware kiểm tra đã đăng nhập  
│   └─ routes/  
│       └─ user.route.js ← Khai báo các route /users/*
```

Khác Bài 10.2 ở chỗ: Bài này đặt tất cả code trong thư mục `src/` (quy ước phổ biến trong project lớn). Script `dev` cũng phải trỏ đến `src/index.js`.

Bước 1: Tạo project (🖥️ TERMINAL)

Mở Terminal, di chuyển tới thư mục muốn lưu project, chạy:

```
# Tạo thư mục project và vào trong
mkdir 10.3.session-middleware
cd 10.3.session-middleware

# Khởi tạo package.json
npm init -y

# Cài 6 package production:
#   express          - framework web
#   mongoose          - kết nối MongoDB
#   bcrypt            - hash mật khẩu (Bài 10.2 đã học)
#   dotenv            - đọc .env
#   express-session   - quản lý session qua cookie
#   connect-mongo     - lưu session vào MongoDB (thay vì RAM)
npm install express mongoose bcrypt dotenv express-session connect-mongo

# Cài nodemon cho dev
npm install nodemon --save-dev

# Tạo cấu trúc thư mục src/
# Flag -p (parents) cho phép tạo nhiều cấp thư mục cùng lúc
mkdir -p src/config src/models src/controllers src/middlewares src/routes
```

Windows PowerShell: Nếu `mkdir -p` không chạy được, tạo từng thư mục riêng: `mkdir src`, rồi `mkdir src\config`, v.v.

Bước 1b: Cấu hình package.json (📄 SỬA FILE)

Mở `package.json`, thêm `"type": "module"` và script `"dev"`. Lưu ý script trỏ tới `src/index.js`:

```
{
  "name": "10.3.session-middleware",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/index.js"
  },
  "dependencies": {
    "bcrypt": "...",
    "connect-mongo": "...",
    "dotenv": "...",
    "express": "...",
    "express-session": "...",
    "mongoose": "..."
  },
  "devDependencies": {
```

```
    "nodemon": "..."  
  }  
}
```

Bước 1c: Tạo file `.env` (📝 TẠO FILE)

Tạo file mới `.env` ở thư mục gốc project, dán nội dung:

```
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/?  
appName=Cluster0  
SESSION_SECRET=my_session_secret_chuoi_ngau_nhien_dai_hon_la_tot
```

Thay thông tin MongoDB và đổi `SESSION_SECRET` thành chuỗi dài ngẫu nhiên (càng dài càng tốt - dùng để ký số cookie).

Tạo file `.gitignore` (📝 TẠO FILE):

```
node_modules  
.env
```

Bước 2: Tạo kết nối database (📝 TẠO FILE)

Tạo file mới `src/config/db.js`, dán nội dung:

```
import mongoose from 'mongoose';  
  
const connectDb = async () => {  
  try {  
    const conn = mongoose.connect(process.env.MONGODB_URI)  
    console.log('Kết nối mongodb thành công.');  } catch (error) {  
    console.error('Lỗi kết nối mongodb: ', error.message);  
  }  
}  
  
export default connectDb
```

Bước 3: Cấu hình Session (📝 TẠO FILE)

Tạo file mới `src/config/session.js`, dán nội dung:

```
// express-session: package quản lý phiên đăng nhập qua cookie.  
import session from 'express-session';  
import dotenv from 'dotenv';  
// connect-mongo: lưu session vào MongoDB (thay vì RAM mặc định).  
import MongoStore from 'connect-mongo'  
dotenv.config()  
  
// session(options) trả về 1 middleware - gắn vào app bằng app.use(sessionConfig).  
const sessionConfig = session({
```

```

// secret: chuỗi bí mật để KÝ SỐ (sign) cookie chứa session ID.
// → Client không sửa được cookie mà không bị phát hiện.
secret: process.env.SESSION_SECRET,

// resave: false → không lưu lại session vào store nếu không thay đổi (tiết kiệm
IO).
resave: false,

// saveUninitialized: false → không tạo session cho request chưa có dữ liệu (vd:
chưa đăng nhập).
// → Giảm số session rác trong DB.
saveUninitialized: false,

cookie: {
  // httpOnly: true → JavaScript ở client KHÔNG đọc được cookie này (chống
XSS).
  // Cookie chỉ được gửi tự động qua HTTP request.
  httpOnly: true,
  // maxAge: thời gian sống của cookie (đơn vị ms).
  // 24 * 60 * 60 * 1000 = 86400000 ms = 24 giờ.
  maxAge: 24 * 60 * 60 * 1000
},

// STORE: nơi lưu session ở phía server.
// Mặc định lưu RAM → mất khi restart server. Dùng MongoStore lưu vào MongoDB
cho persistent.
store: MongoStore.create({
  mongoUrl: process.env.MONGODB_URI,
  collectionName: 'sessions', // Tên collection chứa session
  ttl: 24 * 60 * 60,           // Time-to-live: 24 giờ (đơn vị GIÂY, khác
cookie.maxAge dùng ms)
  autoRemove: 'native'        // Dùng MongoDB TTL index để tự xóa session
hết hạn
})
})

export default sessionConfig

```

Giải thích cấu hình Session:

Option	Giá trị	Ý nghĩa
secret	Chuỗi bí mật	Dùng để ký (sign) session ID, chống giả mạo
resave	false	Không lưu lại session nếu không thay đổi
saveUninitialized	false	Không tạo session cho request chưa đăng nhập
cookie.httpOnly	true	Cookie chỉ gửi qua HTTP, JavaScript phía client KHÔNG đọc được (chống XSS)
cookie.maxAge	86400000 (24h)	Cookie hết hạn sau 24 giờ

store	MongoStore	Lưu session vào MongoDB thay vì bộ nhớ server
ttl	86400 (24h)	Thời gian sống của session trong MongoDB
autoRemove	'native'	MongoDB tự xóa session hết hạn

Tại sao lưu session vào MongoDB? Mặc định `express-session` lưu trong bộ nhớ RAM - nếu restart server thì mất hết session. Lưu vào MongoDB giúp session tồn tại qua các lần restart.

Bước 4: Tạo User Model (📝 TẠO FILE)

Tạo file mới `src/models/user.model.js`, dán nội dung:

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  username: {type: String, require: true, unique: true, trim: true, minlength: [3, "username phải có ít nhất 3 kí tự"]},
  hashedPassword: {type: String, require: true}
}, {timestamps: true})

const User = mongoose.model('User', userSchema)

export default User
```

Bước 5: Tạo Middleware requireLogin (📝 TẠO FILE)

Tạo file mới `src/middlewares/requireLogin.js`, dán nội dung:

```
// MIDDLEWARE BẢO VỆ: chặn request nếu chưa đăng nhập.
// Cách dùng: router.get('/profile', requireLogin, getProfile)
const requireLogin = (req, res, next) => {
  // req.sessionID là ID session (chuỗi dài, do express-session sinh ngẫu nhiên).
  console.log('==== requireLogin middleware ====');
  console.log('SessionID: ', req.sessionID);
  // ?? (Nullish coalescing) - trả về phải nếu vế trái là null/undefined.
  console.log('User: ', req.session.user ?? 'Chưa đăng nhập');

  // Nếu session chưa có user (chưa đăng nhập) → trả 401.
  // return để DỪNG hàm tại đây (không gọi next()).
  if (!req.session.user) {
    return res.status(401).json({error: "Chưa đăng nhập"})
  }

  // Gắn thông tin user vào req để controller phía sau có thể dùng (req.user).
  req.user = req.session.user;
  // Cho request đi tiếp đến route handler chính.
  next();
}

export default requireLogin
```

Giải thích:

- Kiểm tra `req.session.user` - nếu không có nghĩa là chưa đăng nhập.
- `req.user = req.session.user` - Gắn thông tin user vào `req` để các controller sau có thể sử dụng.
- `??` - Nullish coalescing operator, trả về vế phải nếu vế trái là `null / undefined`.

Bước 6: Tạo Controller (👉 TẠO FILE)

Tạo file mới `src/controllers/user.controller.js`, dán nội dung:

```
import bcrypt from 'bcrypt';
import User from '../models/user.model.js';

// ===== ĐĂNG KÝ =====
export const registerUser = async (req, res) => {
  try {
    const {username, password} = req.body

    // BƯỚC 1: Kiểm tra username đã tồn tại chưa.
    // (Mặc dù schema có unique: true, kiểm tra trước để trả lỗi rõ ràng hơn.)
    const existingUser = await User.findOne({username})
    if (existingUser) {
      // 409 Conflict - dữ liệu xung đột với trạng thái hiện tại của server.
      return res.status(409).json({error: "Username đã tồn tại"})
    }

    // BƯỚC 2: Hash mật khẩu rồi lưu vào MongoDB.
    const hashedPassword = await bcrypt.hash(password, 10)
    // User.create() = shortcut cho: new User(...).save()
    const newUser = await User.create({username, hashedPassword})
    res.status(201).json({
      message: "Đăng kí thành công",
      userId: newUser._id // Trả về _id để client biết user vừa tạo
    })

  } catch (error) {
    console.error('Lỗi đăng ký user: ', error.message);
    res.status(500).json({error: 'Lỗi hệ thống đăng kí user'})
  }
}

// ===== ĐĂNG NHẬP =====
export const loginUser = async (req, res) => {
  try {
    const {username, password} = req.body
    const user = await User.findOne({username})
    // ⚠ CẢNH BÁO: nếu user = null, dòng dưới sẽ throw "Cannot read
    'hashedPassword' of null".
    // → NÊN kiểm tra: if (!user) return res.status(401)... trước khi truy cập
    property.
    const storedHash = user.hashedPassword
```

```

    if (!storedHash) {
        return res.status(401).json({message: 'Sai thông tin đăng nhập'})
    }

    // So sánh mật khẩu nhập với hash đã lưu.
    const isMatch = await bcrypt.compare(password, storedHash)
    if (isMatch) {
        // ★ ĐIỂM MẮU CHỐT CỦA SESSION: lưu thông tin user vào req.session.
        // express-session sẽ TỰ ĐỘNG:
        // 1) Lưu req.session vào MongoStore
        // 2) Set cookie 'connect.sid' chứa session ID gửi về client
        // Các request sau, client gửi kèm cookie → server đọc được
        req.session.user = {
            req.session.user = {
                userId: user._id,
                username: user.username
            }
            res.status(200).json({message: 'Đăng nhập thành công', user:
req.session.user, sessionId: req.sessionID})
        }
        else {
            res.status(401).json({message: "Sai thông tin đăng nhập"})
        }
    } catch (error) {
        console.error('Lỗi đăng nhập');
        res.status(500).json({error: 'Lỗi hệ thống đăng nhập'})
    }
}

// ===== XEM PROFILE (đã được bảo vệ bởi requireLogin)
=====
export const getProfile = async (req, res) => {
    try {
        // req.user được middleware requireLogin gán từ req.session.user.
        const user = await User.findById(req.user.userId)

        res.json({
            message: "Lấy profile thành công",
            sessionId: req.sessionID,
            user
        })
    } catch (error) {
        res.status(500).json({error: "Lỗi hệ thống"})
    }
}

// ===== ĐĂNG XUẤT =====
export const logout = async (req, res) => {
    // ?. (Optional chaining) – không lỗi nếu req.session.user là undefined.
    const username = req.session.user?.username;

    // req.session.destroy(callback) – XÓA session khỏi store (MongoDB).

```

```

// Callback nhận err nếu có lỗi.
req.session.destroy((err) => {
  if (err) {
    return res.status(500).json({message: "Lỗi đăng xuất"})
  }
  // res.clearCookie('tên cookie') - XÓA cookie trên trình duyệt client.
  // 'connect.sid' là tên cookie MẶC ĐỊNH của express-session.
  res.clearCookie('connect.sid')
  res.json({message: `${username} đã đăng xuất!`})
})
}

```

Giải thích luồng Session:

1. **Đăng ký** - Hash mật khẩu, lưu user vào DB. Kiểm tra trùng username trước (status 409 Conflict).
2. **Đăng nhập** - So sánh mật khẩu bằng `bcrypt.compare()` . Nếu đúng, lưu thông tin user vào session: `req.session.user = { userId, username }` . Session được tự động lưu vào MongoDB qua `connect-mongo` .
3. **Xem profile** - Middleware `requireLogin` kiểm tra session. Nếu đã đăng nhập, lấy thông tin user từ DB.
4. **Đăng xuất** - `req.session.destroy()` xoá session khỏi MongoDB.
`res.clearCookie('connect.sid')` xoá cookie trên trình duyệt.

Bước 7: Tạo Route (📝 TẠO FILE)

Tạo file mới `src/routes/user.route.js` , dán nội dung:

```

import {Router} from 'express';
import { getProfile, loginUser, logout, registerUser } from
'../controllers/user.controller.js';
import requireLogin from '../middlewares/requireLogin.js';

const router = Router();

// PUBLIC ROUTES (ai cũng truy cập được)
router.post('/register', registerUser)
router.post('/login', loginUser)

// PROTECTED ROUTES (chỉ user đã đăng nhập mới truy cập được)
// → requireLogin chạy TRƯỚC. Nếu chưa đăng nhập → trả 401, không tới handler chính.
router.get('/profile', requireLogin, getProfile)
router.post('/logout', requireLogin, logout)

export default router;

```

`/profile` và `/logout` được bảo vệ bởi `requireLogin` - chỉ user đã đăng nhập mới truy cập được.

Bước 8: Tạo file chính (📝 TẠO FILE)

Tạo file mới `src/index.js` , dán nội dung:

```
import express from 'express';
import dotenv from 'dotenv'
import connectDb from './config/db.js';
import userRoute from './routes/user.route.js'
import sessionConfig from './config/session.js';

dotenv.config()
await connectDb() // Đợi DB kết nối xong rồi mới start server

const app = express()
app.use(express.json())

// GẮN MIDDLEWARE SESSION cho TOÀN APP.
// Sau dòng này, MỌI request đều có req.session, req.sessionID.
// PHẢI gắn TRƯỚC các route sử dụng session.
app.use(sessionConfig)

app.use('/users', userRoute)

app.listen(3000, () => {
  console.log('Server is running ...');
})
```

Bước 9: Chạy server (🖥️ TERMINAL)

Đảm bảo bạn đang ở thư mục gốc `10.3.session-middleware/`, chạy:

```
npm run dev
```

Khi server khởi động thành công, Terminal hiện:

```
[nodemon] starting `node src/index.js`
Kết nối mongodb thành công.
Server is running ...
```

Giữ Terminal này chạy. Mở Terminal MỚI để test ở Bước 10.

Bước 10: Test luồng đăng nhập (🖥️ TERMINAL MỚI)

Mở **Terminal mới** (giữ Terminal cũ đang chạy server). Bài này test theo LUỒNG 5 bước - phải làm tuần tự.

Test 1 - Đăng ký user mới:

```
curl -X POST http://localhost:3000/users/register \
-H "Content-Type: application/json" \
-d '{"username": "student01", "password": "123456"}'
```

Kết quả: `{"message": "Đăng kí thành công", "userId": "..."}"`

Test 2 - Đăng nhập, LƯU cookie vào file `cookies.txt` :

```
curl -X POST http://localhost:3000/users/login \
  -H "Content-Type: application/json" \
  -d '{"username": "student01", "password": "123456"}' \
  -c cookies.txt
```

Flag `-c cookies.txt` = "**cookie jar**" - bảo curl LƯU các cookie server gửi về vào file này. Sau khi chạy, có thể thấy file `cookies.txt` mới được tạo trong thư mục hiện tại.

Kết quả: `{"message": "Đăng nhập thành công", "user": {...}, "sessionId": "..."}"`

Test 3 - Xem profile, GỬI KÈM cookie từ file:

```
curl http://localhost:3000/users/profile -b cookies.txt
```

Flag `-b cookies.txt` = "**browser cookie**" - bảo curl GỬI cookie từ file kèm request. Nhờ vậy server biết "đây là user vừa login ở Test 2".

Kết quả: `{"message": "Lấy profile thành công", "sessionID": "...", "user": {"_id": "...", "username": "student01", ...}}`

Test 4 - Đăng xuất:

```
curl -X POST http://localhost:3000/users/logout -b cookies.txt
```

Kết quả: `{"message": "student01 đã đăng xuất!"}`

Test 5 - Xem profile sau khi đã đăng xuất (PHẢI bị từ chối):

```
curl http://localhost:3000/users/profile -b cookies.txt
```

Kết quả: `{"error": "Chưa đăng nhập"}` (status 401)

🟢 Nếu Test 5 trả về 401 nghĩa là cơ chế session + middleware hoạt động đúng: cookie cũ không còn hiệu lực sau logout.

Test bằng Postman / Thunder Client (tùy chọn)

Nếu KHÔNG dùng curl, bạn có thể test bằng Postman hoặc Thunder Client:

Bước	Method	URL	Body
1	POST	<code>http://localhost:3000/users/register</code>	<code>{"username": "student01", "password": "123456"}</code>
2	POST	<code>http://localhost:3000/users/login</code>	<code>{"username": "student01", "password": "123456"}</code>
3	GET	<code>http://localhost:3000/users/profile</code>	(không)
4	POST	<code>http://localhost:3000/users/logout</code>	(không)
5	GET	<code>http://localhost:3000/users/profile</code>	(không)

Postman/Thunder Client TỰ ĐỘNG quản lý cookie - không cần config gì thêm. Sau khi login, các request sau tự gửi cookie kèm.

Dừng server: Quay lại Terminal đang chạy server, nhấn `Ctrl + C`.

Lỗi thường gặp

Lỗi	Nguyên nhân	Cách sửa
Error: secret option required for sessions	Thiếu SESSION_SECRET trong .env	Thêm SESSION_SECRET=chuoai_bi_mat vào file .env
Cannot find module 'connect-mongo'	Chưa cài	<code>npm install connect-mongo</code>
Profile trả về 401 dù đã login	Không gửi cookie kèm request	Dùng <code>-b cookies.txt</code> với curl hoặc bật "Send cookies" trong Postman
Session mất sau restart	Kiểm tra MongoStore có kết nối đúng	Kiểm tra mongoUrl trong session config trả đúng MongoDB Atlas
req.session.user là undefined	Session chưa được tạo (chưa đăng nhập) hoặc cookie hết hạn	Đăng nhập lại